

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_3e301d0b-e59\agent-  
a8a4aebf800b942e7.jsonl`  
- SHA-256: `8a7fa336a18466fbb28e7a089cbc539b846812d7fc32ef3b3be8666d9ece2c0e`  
- Source modified: `2026-06-13T18:52:14+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_3e301d0b-e59`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T18:49:16.862Z)

You are designing fusion P2 for a badminton rally detector. Synthesize a CONCRETE, buildable implementation plan from these subsystem maps (JSON): [{"findings":"## 4-Phase Flow (healthcheck ? P2 trajectory?windows ? P3 handoff ? P4 DB/link)\n\n**Phase 1: Healthcheck Gate** (trajectory_hybrid.py:140-145)\n- `runner.healthcheck()` returns `(bool, str)` at line 141-142\n- For TrackNet: validates WSL env, TensorFlow imports, GPU visibility (tracknet_runner.py:97-116)\n- For WASB: validates WSL env, PyTorch imports, CUDA availability (wasb_runner.py:66-80)\n- On failure, logs error and returns empty list; passes through detection_params dict\n\n**Phase 2: Trajectory?Candidate Windows** (trajectory_hybrid.py:147-193)\n- Line 152: `csv\_path = runner.run\_predict(video\_path, out\_dir, video\_id=video\_id)` produces CSV\n- Line 157: `points = runner.parse\_trajectory\_csv(csv\_path)` parses Frame,Visibility,X,Y columns\n- Line 161-165: `runner.trajectory\_to\_action\_windows(points, fps, frame\_width, chunk\_start=0.0, velocity\_thresh, merge\_gap, min\_window\_duration, chunk\_index=0)` seeds windows from trajectory\n- Window merging: groups active frames by `merge\_gap` (dead_time_merge_gap=2.0 default at line 125), filters by `min\_window\_duration` (default 2.0s)\n- **Persistence call (lines 176-190)**: `self.db.add\_candidate\_segment(video\_id, detector=self.name, start\_time=cw[\"start\"], end\_time=cw[\"end\"], chunk\_index=cw[\"chunk\_index\"], confidence=min(1.0, cw[\"peak\_velocity\"])), stats=stats, detection\_params=detection\_params)`\n- **Stats JSON schema written** (lines 179-184):\n {\n \"peak_velocity\": float,\n \"mean_velocity\": float,\n \"active_frame_count\": int,\n \"track_id_count\": int # 1 for trajectory (shuttle), N for person detector\n }\n\n**Phase 3: AI Provider Handoff** (trajectory_hybrid.py:195-236)\n- Line 196-197: extracts padded clips from candidates (padding=handoff_padding=2.0s default)\n- Line 201-224: `process\_candidate\_window()` calls `provider.analyze\_video()` over each padded clip\n- Line 214-216: embeds candidate_ids into segments for later linking\n- Line 227-231: concurrent ThreadPoolExecutor (ai_max_workers=5 default) submits all windows\n- Returns AI-confirmed segments with adjusted timestamps (line 214-215: `seg[\"start\_time\"]/seg[\"end\_time\"] = float(...) + w\_start`)\n\n**Phase 4: DB Population + Linking** (trajectory_hybrid.py:238-262)\n- Line 250: `segment\_id = self.db.add\_play\_segment(video\_id, start, end, None, None, metadata)` with metadata dict\n- Line 252-254: `self.db.link\_candidate\_to\_segment(candidate\_id, segment\_id)` wires raw detection to confirmed segment\n- Returns segments_data list with id, start_time, end_time, duration, metadata (shot_count, shot_count_confidence, detector tag)\n\n## Trajectory-to-Windows Seed Method\n\n**Exact location**: `TrackNetRunner.trajectory\_to\_action\_windows()` at tracknet_runner.py:252-319 (reused by WasbRunner via staticmethod alias at wasb_runner.py:178-179)\n\n**Algorithm**:\n1. Line 277-290: Computes per-frame normalized velocity from visible points: `velocity = (dist / frame\_width) \* (fps / dframes)` where dist is Euclidean distance, compared to `velocity\_thresh` (default 0.02)\n2. Line 295: `max\_gap\_frames = int(merge\_gap \* fps)` groups active frames\n3. Line 310-317: Groups consecutive high-velocity frames with gap < max_gap_frames using `\_finalize(group)`\n4. Line 319: Filters windows by minimum duration: `[w for w in windows if (w[\"end\"] - w[\"start\"]) >= min\_window\_duration]`\n5. Returns List[Dict] with keys: start, end, active_frame_count, peak_velocity, mean_velocity, track_id_count, chunk_index\n\n## Candidate Persistence Contract\n\n**Exact call site**: trajectory_hybrid.py:185-190\n```\npython\ncandidate_ids[i] = self.db.add_candidate_segment(\n video_id, detector=self.name,\n start_time=cw[\"start\"],

```

end_time=cw["end"],\n      chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),\n      stats=stats,
detection_params=detection_params,\n)\n```\n\n**Database schema**
(backend/infrastructure/database.py:97-121):\n- Table: candidate_segments with columns: id
(AUTOINCREMENT), video_id, detector, chunk_index, start_time, end_time, duration, confidence,
stats (JSON), detection_params (JSON), confirmed_segment_id, human_label, notes, created_at\n-
Insertion (database.py:250-256): `INSERT INTO candidate_segments (video_id, detector,
chunk_index, start_time, end_time, duration, confidence, stats, detection_params) VALUES (?, ?,
?, ?, ?, ?, ?, ?)`\n- Returns `cursor.lastrowid` for linkage\n\n**Stats JSON schema
persisted** (trajectory_hybrid.py:179-184):\n```\njson\n{\n  \"peak_velocity\": float,\n  \"mean_velocity\": float,\n  \"active_frame_count\": int,\n  \"track_id_count\":
int\n}\n```\n\n**Detection params** (trajectory_hybrid.py:132-138):\n```\njson\n{\n  \"detector\":
str (self.name = \"tracknet_hybrid\" or \"wasb_hybrid\"),\n  \"velocity_thresh\": float,\n  \"merge_gap\": float,\n  \"min_action_window_duration\": float,\n  \"weights_path\": str,\n  \"queue_length\": int (TrackNet only) or omitted (WASB)\n}\n```\n\n## Subclass Contract
(TrajectoryHybridSegmenter ABC)\n\n**Family** (trajectory_hybrid.py:49 ? config section under
`indexing.*`, output subdir, metadata detector-tag prefix\n- WasbHybridSegmenter.family =
\"wasb\" (wasb_hybrid.py:24)\n- TrackNetHybridSegmenter.family = \"tracknet\"
(tracknet_hybrid.py:22)\n\n**Display Label** (trajectory_hybrid.py:51 ? human label for log
lines\n- WasbHybridSegmenter.display_label = \"WASB\" (wasb_hybrid.py:25)\n-
TrackNetHybridSegmenter.display_label = \"TrackNet\" (tracknet_hybrid.py:23)\n\n**Name &
Description** (BasePlaySegmenter contract (base.py:12-21)\n- `name` property: unique identifier
(\"wasb_hybrid\", \"tracknet_hybrid\")\n- `description` property: human-readable
explanation\n\n**_build_runner() contract** (trajectory_hybrid.py:53-55):\n- Returns
`Tuple[DetectorRunner, Dict[str, Any]]` where second element is detector-specific
detection_params extras\n- WasbHybridSegmenter (wasb_hybrid.py:36-38): returns
`(WasbRunner(cfg), {\"weights_path\": cfg.weights_path})`\n- TrackNetHybridSegmenter
(tracknet_hybrid.py:34-39): returns `(TrackNetRunner(cfg), {\"weights_path\": cfg.weights_path,
\"queue_length\": cfg.queue_length})`\n\n**DetectorRunner ABC contract** (base.py:141-168):\n-
`run_predict(video_win_path: str, output_win_dir: str, video_id: Optional[str] = None) ->
Optional[str]`: returns Windows path to trajectory CSV or None\n- `healthcheck() -> Tuple[bool,
str]`: returns (ok, message) for env readiness\n- **De facto trajectory methods** (duck-typed,
not abstract):\n  - `parse_trajectory_csv(csv_path: str) -> List[TrajectoryPoint]`\n  -
`trajectory_to_action_windows(...) -> List[Dict[str, Any]]`\n\n## Precise Seam for
FusionSegmenter\n\n**FusionSegmenter should extend TrajectoryHybridSegmenter**
(trajectory_hybrid.py:44-56) because:\n\n1. **Composition path** (Override `_build_runner()`) to
return a custom runner that:\n  - Accepts TrajectoryHybridSegmenter's existing DetectorRunner
contract (line 53-55)\n  - Can wrap or chain runners (e.g., execute TrackNet, then apply
rally_gate.py filtering at Phase 2)\n  - Can substitute person-detection producer
(person_detector.py:138-254) inline or post-filter\n\n2. **Rally Gate Seam (Gate A, line 16 of
rally_gate.py)**:\n  - `gate_windows(points, windows, fps, min_crossings=2,
deadband_frac=0.06) -> List[GatedWindow]` filters trajectory windows by net-crossing count\n  -
Can be applied post-`trajectory_to_action_windows()` within Phase 2
(trajectory_hybrid.py:161-165 would call the filtered windows)\n  - Stateless gate:
`apply_gate(points, windows, fps, min_crossings, deadband_frac) -> List[Interval]` returns kept
windows\n\n3. **Person Cue Seam (person_detector.py:48-254)**:\n  -
`PersonDetector.extract_action_windows_from_chunk()` produces identical window schema (start,
end, active_frame_count, peak_velocity, mean_velocity, track_id_count, chunk_index)\n  -
Integrates via `_build_runner()` ? returns a runner that internally calls both trajectory +
person detector, merges/intersects windows, or seeds AI handoff over person-motion regions\n  -
Person stats have `track_id_count > 1` (multiple persons); trajectory has `track_id_count =
1`\n\n4. **Composition option - FusionRunner pattern**:\n  - python\n    class
FusionSegmenter(TrajectoryHybridSegmenter):\n      family = \"fusion\"\n      display_label =
\"Fusion\"\n      def _build_runner(self, idx_cfg):\n        # Option A: return a
runner that internally composes\n        return FusionRunner(\n
trajectory_runner=TrackNetRunner(...),\n
person_detector=PersonDetector(...),\n
gate_config={...}\n
), {fusion-specific params}\n
# Phase 2 override
(if needed):\n
# all_candidate_windows = fusion_runner.trajectory_to_action_windows(...)\n
# # then apply rally_gate or intersect with person windows\n
```\n\n**Key plugging
points**:\n- **Line 161-165** (trajectory_hybrid.py): `trajectory_to_action_windows()` call ?
output can be post-filtered or replaced\n- **Line 176-190** (add_candidate_segment())`
persistence ? FusionRunner can enrich stats dict with composite/person metrics\n- **Line
201-224** (process_candidate_window()) in Phase 3 ? candidate_ids from Phase 2 are threaded
through to Phase 4 linking, \"anchors\": [\"backend/pipeline/segmenters/trajectory_hybrid.py:44-
56\", \"backend/pipeline/segmenters/trajectory_hybrid.py:140-

```

145", "backend/pipeline/segmenters/trajectory\_hybrid.py:147-193", "backend/pipeline/segmenters/trajectory\_hybrid.py:152", "backend/pipeline/segmenters/trajectory\_hybrid.py:157", "backend/pipeline/segmenters/trajectory\_hybrid.py:161-165", "backend/pipeline/segmenters/trajectory\_hybrid.py:176-190", "backend/pipeline/segmenters/trajectory\_hybrid.py:179-184", "backend/pipeline/segmenters/trajectory\_hybrid.py:195-236", "backend/pipeline/segmenters/trajectory\_hybrid.py:238-262", "backend/pipeline/segmenters/trajectory\_hybrid.py:250", "backend/pipeline/segmenters/trajectory\_hybrid.py:252-254", "backend/pipeline/segmenters/wasb\_hybrid.py:23-38", "backend/pipeline/segmenters/tracknet\_hybrid.py:21-39", "backend/pipeline/detectors/base.py:141-168", "backend/pipeline/detectors/base.py:53-55", "backend/pipeline/detectors/tracknet\_runner.py:232-249", "backend/pipeline/detectors/tracknet\_runner.py:252-319", "backend/pipeline/detectors/tracknet\_runner.py:277-290", "backend/pipeline/detectors/tracknet\_runner.py:310-317", "backend/pipeline/detectors/wasb\_runner.py:62-64", "backend/pipeline/detectors/wasb\_runner.py:178-179", "backend/pipeline/detectors/rally\_gate.py:16", "backend/pipeline/detectors/rally\_gate.py:91-112", "backend/pipeline/detectors/person\_detector.py:48-254", "backend/pipeline/detectors/person\_detector.py:138-254", "backend/infrastructure/database.py:97-121", "backend/infrastructure/database.py:240-261", "backend/infrastructure/database.py:250-256", "backend/pipeline/segmenters/base.py:6-33", "backend/pipeline/segmenters/base.py:12-21"]

"risks":["Candidate seeding from Phase 2 windows uses single `chunk\_index=0` for live processing (trajectory\_hybrid.py:164); FusionSegmenter must preserve this or explicitly set per merged/filtered window. If chunk-based processing is later enabled, loop over chunks with different chunk\_index values."], "Stats JSON schema is detector-specific (trajectory has single track\_id\_count=1 always; person detector has variable count). Fusion must handle both schemas or normalize on insertion. See trajectory\_hybrid.py:183 vs person\_detector.py:236."], "Rally Gate operates on trajectory points post-parsing; if Gate A is integrated into FusionRunner.trajectory\_to\_action\_windows(), it must preserve the TrajectoryPoint objects across the filtering call, or re-filter the original points. Currently rally\_gate.py:91 expects full points list (not just window intervals).", "Phase 3 AI handoff uses `candidate\_ids[wi]` which is a list indexed by window order (trajectory\_hybrid.py:216). If FusionSegmenter merges/deduplicates windows from multiple sources, the candidate\_ids indexing breaks. Solution: map each output window back to its source candidate ID(s) or generate fusion-specific IDs in add\_candidate\_segment."], {"findings":["rally\_gate.py Mapping for Production Integration\n\n#### Function Signatures (exact from backend/pipeline/detectors/rally\_gate.py)\n\n\*\*estimate\_play\_axis(points) -> Tuple[str, float, float]\*\* (lines 55-66)\n- Returns (axis 'x'|'y', net\_value, span) from visible points\n- axis: the image dimension with larger shuttle spread\n- net\_value: median shuttle coordinate on that axis (where net is located)\n- span: p95-p5 spread of coordinates (robust outlier tolerance)\n\n\*\*count\_net\_crossings(window\_points, axis: str, net: float, deadband: float) -> int\*\* (lines 69-84)\n- Counts sign changes of (coord - net) across visible points\n- Ignores deadband zone (abs(coord - net) < deadband) to suppress jitter\n- Returns integer crossing count\n\n\*\*gate\_windows(points, windows: List[Interval], fps: float, min\_crossings: int = 2, deadband\_frac: float = 0.06) -> List[GatedWindow]\*\* (lines 91-105)\n- Takes list of (start\_sec, end\_sec) tuples in windows parameter\n- Returns List[GatedWindow] with decision on each\n- Estimates axis/net once over full trajectory (stable), applies to all windows\n- deadband\_frac: fraction of axis span to use as deadband (default 6%)\n- min\_crossings: default value is 2 (line 92)\n\n\*\*apply\_gate(points, windows: List[Interval], fps: float, min\_crossings: int = 2, deadband\_frac: float = 0.06) -> List[Interval]\*\* (lines 108-112)\n- Convenience wrapper: returns only windows that pass gate\n- Returns filtered list of (start\_sec, end\_sec) tuples\n\n\*\*GatedWindow dataclass\*\* (lines 28-34)\n- Fields: start: float, end: float, crossings: int, visible\_points: int, kept: bool\n- kept = (crossings >= min\_crossings) per line 104\n\n#### Caller Adaptation Pattern\n\n\*\*distill\_local.py line 56:\*\*\n\n```\npython\ngated = rally\_gate.gate\_windows(points, intervals, fps, min\_crossings=0)\n```\n\nCalls with min\_crossings=0 (disabled) to extract features only.\n\n\*\*calibrate\_local.py lines 50-52:\*\*\n\n```\npython\nwins = [(w["start"], w["end"]) for w in windows] # rich window dict -> (start,end) tuple\nif mc > 0:\n wins = rally\_gate.apply\_gate(points, wins, fps, min\_crossings=mc)\n```\n\n- trajectory\_to\_action\_windows (line 46-49) returns List[Dict[str, Any]] with keys: start, end, active\_frame\_count, peak\_velocity, mean\_velocity, track\_id\_count, chunk\_index (tracknet\_runner.py lines 299-307)\n\n\*\*ADAPTATION:\*\* Extract (start, end) tuples from rich dict (line 50): [(w["start"], w["end"]) for w in windows]\n\n- Pass tuples to rally\_gate.apply\_gate() expecting Interval (Tuple[float, float])\n\n\*\*Default OFF behavior (line 36, 51-52):\*\* min\_crossings defaults to 0; if mc > 0, apply gate, else skip\n\n#### Default

min\_crossings Value\n\n\*\*Default: 2\*\* per rally\_gate.py lines 92 and 108 function signatures.\nHowever, \*\*production baseline uses 0\*\* per calibrate\_local.py line 36:\n``python\nBASELINE\_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0) # today's local default (windowing, no gate)\n``\n\nThis means the gate is OFF by default (min\_crossings=0 = byte-identical to not calling rally\_gate at all).\n\n### Minimal Production Integration Pattern\n\n\*\*For live trajectory windowing with new min\_crossings knob:\*\*\n\n1. Call trajectory\_to\_action\_windows() normally (returns rich window dicts)\n2. Extract tuples: intervals = [(w["start"], w["end"]) for w in windows]\n3. Conditionally apply gate:\n``python\nif min\_crossings > 0:\n intervals = rally\_gate.apply\_gate(points, intervals, fps, min\_crossings=min\_crossings)\n``\n\n4. Use filtered intervals downstream\n\n\*\*Knob default:\*\* min\_crossings = 0 (OFF, preserves current behavior exactly)\n\n\*\*Optional tuning range:\*\* 0-3 per calibrate\_local.py line 63 grid search\n\n### Key Implementation Details\n\nrally\_gate.py is pure functions only (line 18: "Pure functions only ? no I/O, unit-tested")\n\nExpects TrajectoryPoint objects with .frame, .visible, .x, .y attributes (tracknet\_runner.py lines 80-84)\n\nfps parameter required for frame-to-second conversion (lines 87-88)\n\naxis/net/deadband estimated once per apply\_gate/gate\_windows call over entire trajectory, then reused for all windows\n\nWindow intervals passed as List[Tuple[float, float]] where floats are seconds (not frames)\n\nDeadband = deadband\_frac \* span ensures scale invariance (line 97)\n\n", "anchors": ["backend/pipeline/detectors/rally\_gate.py:55-66 (estimate\_play\_axis)", "backend/pipeline/detectors/rally\_gate.py:69-84 (count\_net\_crossings)", "backend/pipeline/detectors/rally\_gate.py:91-105 (gate\_windows)", "backend/pipeline/detectors/rally\_gate.py:108-112 (apply\_gate)", "backend/pipeline/detectors/rally\_gate.py:28-34 (GatedWindow)", "backend/eval/distill\_local.py:56 (min\_crossings=0 usage)", "backend/eval/calibrate\_local.py:50-52 (tuple extraction and conditional gating)", "backend/eval/calibrate\_local.py:36 (BASELINE\_LOCAL min\_crossings=0)", "backend/pipeline/detectors/tracknet\_runner.py:252-319 (trajectory\_to\_action\_windows dict shape)", "backend/pipeline/detectors/tracknet\_runner.py:80-84 (TrajectoryPoint definition)"], "risks": ["If min\_crossings > 0 is wired into production but not exposed as a tunable config knob, users cannot turn off the gate if false negatives emerge.", "The gate expects fps to be exact; incorrect fps will cause wrong frame-to-second conversion and wrong deadband calibration.", "The rich window dict from trajectory\_to\_action\_windows has both 'start' and 'end' as seconds (not frames), but the caller must extract them before passing to rally\_gate.", "Deadband is not tunable in the live pipeline (hardcoded to 0.06 of span); if shuttle-specific camera angles need different jitter tolerance, that requires a code change."]], {"findings": "### Config Structure and Validation Pattern\n\n\*\*Root Model:\*\* `AppConfig` (backend/config/models.py:78-237) uses Pydantic with `extra="allow"` (line 201) to tolerate future/legacy keys gracefully. Nested sections are typed as `Field(default\_factory=...)` instances.\n\n\*\*IndexingConfig Structure\*\* (backend/config/models.py:78-128):\n\n- Top-level scalar fields: `default\_segmenter` (str, default="yolo\_hybrid"), `use\_gpu` (bool), `motion\_threshold`, `min\_segment\_duration`, `dead\_time\_merge\_gap`, `chunk\_duration`, `chunk\_overlap`, `detector\_model`, `detector\_score\_threshold`, `yolo\_velocity\_threshold`, `yolo\_frame\_skip`, `yolo\_tracker`, `yolo\_prefetch\_workers`, `min\_action\_window\_duration`, `log\_yolo\_candidates`, `store\_yolo\_candidates`, `ai\_handoff\_padding`, `ai\_max\_workers`, `ai\_chunk\_scale\_width`, `debug\_duration` (Optional[float]).\n\n- Nested configs as factory defaults:\n\n - `tracknet`: TrackNetConfig = Field(default\_factory=TrackNetConfig) (line 107)\n - `wasb`: WasbIndexConfig = Field(default\_factory=WasbIndexConfig) (line 108)\n\n- Validation: `@field\_validator` on `default\_segmenter` (lines 110-115); `@model\_validator(mode="after")` enforces `chunk\_overlap < chunk\_duration` (lines 117-127).\n\n\*\*Nested Sub-Config Classes:\*\*\n\n1. \*\*TrackNetConfig\*\* (backend/config/models.py:33-52):\n\n - `wsl\_distro`, `repo\_dir`, `conda\_sh`, `conda\_env`, `weights\_path`, `queue\_length`, `stage\_in\_wsl`, `wsl\_stage\_dir`, `timeout\_sec`, `velocity\_threshold`, `keep\_frames`, `min\_free\_gb`.\n\n - NO `from\_indexing\_cfg` method here (that's in the dataclass at backend/pipeline/detectors/tracknet\_runner.py:31-60).\n\n2. \*\*WasbIndexConfig\*\* (backend/config/models.py:54-76):\n\n - `wsl\_distro`, `repo\_dir`, `conda\_sh`, `conda\_env`, `weights\_path`, `sport`, `wsl\_stage\_dir`, `timeout\_sec`, `velocity\_threshold`, `keep\_frames`, `min\_free\_gb`.\n\n - Comment at line 55-59 explicitly notes: "Mirrors backend/pipeline/detectors/wasb\_runner.py:WasbConfig.from\_indexing\_cfg so these knobs actually take effect ? previously this whole block was silently dropped because nested config sections defaulted to extra='ignore'".\n\n\*\*Config Loading Behavior\*\* (backend/config/loader.py):\n\n- `load\_config()` (lines 53-88): merges built-in defaults with JSON file via `\*\*get\_builtin\_defaults()`, `\*\*file\_data` then passes to `AppConfig.model\_validate()`.\n\n- `\_unknown\_key\_paths()` (lines 25-42): recursively detects typo'd keys at all nesting levels (top-level extras are kept-but-unread by extra="allow"; section keys are silently dropped if unknown) and logs a warning with dotted paths like

```

`\"indexing.velocity_threshold\"` or `\"validation.relevance.unknown_gate\"` (test case at
backend/config/test_config.py:69-86).\n\n**Config Consumption Pattern** (observed across
codebase):\n- Direct typed access: `cfg.indexing.default_segmenter`,
`cfg.indexing.tracknet.velocity_threshold` (test line 117; main.py line 598 still uses legacy
.get pattern).\n- Legacy dict .get pattern: `self.config.get(\"indexing\",
{})`.\n- .get(\"motion_threshold\", 0.08)` (backend/main.py:598, person_detector.py:71, etc.).\n-
Dataclass bridge: WasbRunner/TrackNetRunner use `WasbConfig.from_indexing_cfg(dict)` /
`TrackNetConfig.from_indexing_cfg(dict)` to build old-style dataclass from config dict
(wasb_runner.py:45-59; tracknet_runner.py:46-60). These read the dict form via `.get()` with
fallback defaults ? **these will NOT see typed defaults if called on a dict**.\n\n**Key Design
Note:** `model_config = {\"extra\": \"allow\", \"validate_assignment\": True}` on AppConfig
(line 200-202) allows graceful forward compat but means typo'd keys silently propagate. The
warning system in loader.py:unknown_key_paths() is the guard.\n\n## Exact Additions
Required\n\n### 1. Add `min_crossings: int = 0` to IndexingConfig\n\n**File:**
backend/config/models.py\n\n**Location:** After line 94 (after `yolo_prefetch_workers: int =
2`)\n\n**Code snippet:**\n\n``python\nmin_action_window_duration: float = 2.0\nlog_yolo_candidates:
bool = True\nstore_yolo_candidates: bool = True\nnai_handoff_padding: float =
2.0\nnai_max_workers: int = 5\nmin_crossings: int = 0 # <-- ADD HERE\nnai_chunk_scale_width: int
= 1280\n```\n\n### 2. Add FusionConfig subclass and integrate into IndexingConfig\n\n**File:**
backend/config/models.py\n\n**Location:** Insert new class after WasbIndexConfig (after line 76),
before IndexingConfig (line 78)\n\n**New Class (FusionConfig):**\n\n``python\nclass
FusionConfig(BaseModel):\n \"\"\"Fusion-specific geometric and player constraints for court
analysis.\n\n Optional ? when omitted, fusion stage uses no court polygon masking,\n
infers net position from video, and expects the sport's default player count.\n\n \"\"\"\n\n court_polygon: Optional[List[List[float]]] = Field(\n default=None,\n
description=\"Optional court boundary as normalized [x, y] points (0-1 range). \"\n\n
\"None = no polygon mask (full-frame analysis). Each point is [x_norm, y_norm].\"\n\n)\n
net_axis: Literal[\"x\", \"y\"] = Field(\n default=\"y\", \n description=\"Net runs
along this axis ('x'=vertical net, 'y'=horizontal net).\"\n\n)\n net_position: float =
Field(\n default=0.5, \n ge=0.0, \n le=1.0, \n description=\"Net
position on the net_axis (0-1 normalized).\"\n\n)\n expected_players: int = Field(\n
default=2, \n ge=1, \n description=\"Expected number of players in frame.\"\n\n
)\n\n```\n\n**Modification to IndexingConfig (line 78+):**\n\nAdd after the `wasb` field (line
108):\n\n``python\n wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)\n
fusion: FusionConfig = Field(default_factory=FusionConfig) # <-- ADD THIS LINE\n\n```\n\n##
Extra/Forbid Behavior Notes\n\n- **No `extra='forbid'` on nested configs:** TrackNetConfig,
WasbIndexConfig, and (new) FusionConfig have implicit `extra='ignore'` (Pydantic default). They
will silently drop unknown keys in their sections. This is intentional for forward compat
(comment at line 55-59 warns this was a silent failure mode).\n- **Top-level AppConfig has
`extra='allow'`** (line 201): unknown top-level keys are kept but unread; the warning system
flags them.\n- **No aliases used:** BaseModel fields use standard names; no `Field(alias=...)`
needed.\n- **Validation assignment enabled:** `validate_assignment: True` means runtime
assignment like `cfg.fusion.net_position = 0.3` will validate the new value.\n\n## Config JSON
Format Examples\n\n**For min_crossings (scalar in indexing root):**\n\n``json\n{\n \"indexing\":
{\n \"min_crossings\": 5\n }\n}\n```\n\n**For fusion sub-config:**\n\n``json\n{\n
\"indexing\": {\n \"fusion\": {\n \"court_polygon\": [[0.1, 0.1], [0.9, 0.1], [0.9,
0.9], [0.1, 0.9]],\n \"net_axis\": \"y\", \n \"net_position\": 0.5, \n \"expected_players\": 2\n }\n}\n}\n```\n\nIf any fusion field is omitted, its default
applies. If the entire `fusion` block is omitted, `FusionConfig()` (all defaults) is used.\", \"anchors\": [\"backend/config/models.py:78-128\", \"backend/config/models.py:33-52\", \"backend/config/models.py:54-76\", \"backend/config/models.py:200-202\", \"backend/config/loader.py:25-42\", \"backend/config/loader.py:53-88\", \"backend/config/test_config.py:69-86\", \"backend/config/test_config.py:141-163\", \"backend/pipeline/detectors/wasb_runner.py:45-59\", \"backend/pipeline/detectors/tracknet_runner.py:46-60\"], \"risks\": [\"The new min_crossings and fusion fields will NOT be validated by existing dataclass adapters (TrackNetConfig.from_indexing_cfg, WasbConfig.from_indexing_cfg) ? they read dict via .get() and won't see typed defaults. If fusion geometry should feed the WASB/TrackNet runners, those adapters need updating to call `idx_cfg.get('fusion', {})` or move to typed access.\", \"FusionConfig uses Optional[List[List[float]]] for court_polygon; JSON null/omitted both work, but callers must handle None explicitly (no empty-list default). Code reading fusion.court_polygon should check `if cfg.fusion.court_polygon is not None` before iteration.\", \"Nested config sections (FusionConfig, TrackNetConfig, WasbIndexConfig) silently drop unknown keys (extra='ignore' is implicit). No validation error if someone mistypes 'net_postition' ? it just won't load. The warning system only catches top-level

```

```

typos."}],{"findings":"### Person-Feature Computation: Pure Function Design Spec\n\n### Context &
Data Shapes\n\n**Input per shuttle-seeded window** (C:\\Users\\avidu\\Projects\\badminton-
highlight-indexer\\backend\\pipeline\\detectors\\person_detector.py:98?136):\n- Per-frame person
detections: `List[(track_id: int, bbox: (x1, y1, x2, y2))` in pixels\n- Per-frame shuttle
(optional): `(x: float, y: float, visible: bool)` from CSV (tracknet_runner.py:80?84)\n- Video
metadata: `fps: float`, `frame_width: float`\n- Court polygon (optional, normalized 0?1):
`List[(norm_x, norm_y)]` closed polygon\n- Net line (optional, normalized 0?1): `(axis: 'x'|'y',
net_coord: float)`\n\n### Design: Scale/Translation-Invariant Features\n\nAll formulas use
frame-width normalization (matching geometry.py:23?40) or **ratios/counts** ? never raw
pixels. This ensures features transfer across resolutions and camera positions.\n\n--\n\n###
Feature 1: `players_in_court`\n\n**Type:** Count aggregation | **Rally signal:** ?2 (singles) / 4
(doubles); 0?1 = dead time\n\n``python\ndef players_in_court(per_frame_in_court_counts:
List[int]) -> float:\n \"\"\"Median count of persistent in-court persons per frame.\n \n
Args:\n per_frame_in_court_counts: List where index is frame; value is count of\n
track IDs detected inside court polygon that frame.\n \n
Returns:\n Median count across all frames in the window. If no frames, return 0.0.\n
\n Scale-invariant: Yes (count, no pixels).\n Edge cases:\n - empty list ? 0.0\n
- single frame ? that frame's count\n - all zeros ? 0.0 (dead time correctly labeled)\n
\n \"\"\"\n if not per_frame_in_court_counts:\n return 0.0\n sorted_counts =
sorted(per_frame_in_court_counts)\n n = len(sorted_counts)\n if n % 2:\n return
float(sorted_counts[n // 2])\n return (float(sorted_counts[n // 2 - 1]) +
float(sorted_counts[n // 2])) / 2.0\n``\n\n### Helper: `point_in_polygon`\n\n``python\ndef
point_in_polygon(point: Tuple[float, float], \n polygon: List[Tuple[float,
float]]) -> bool:\n \"\"\"Ray-casting algorithm for point-in-polygon (all coords normalized
0?1).\n \n Args:\n point: (norm_x, norm_y) where 0 ? x, y ? 1\n polygon:
Closed polygon as list of (norm_x, norm_y) vertices (0?1).\n \n Last vertex is
implicitly connected to first.\n \n Returns:\n True if point is inside or on
boundary.\n \n Scale-invariant: Yes (normalized coordinates).\n Edge case: polygon
None ? False (can't contain; return 0 persons in-court).\n \n \"\"\"\n if not polygon:\n
return False\n x, y = point\n n = len(polygon)\n inside = False\n for i in
range(n):\n x1, y1 = polygon[i]\n x2, y2 = polygon[(i + 1) % n]\n \n
Ray casting: horizontal ray to the right\n if ((y1 <= y < y2) or (y2 <= y < y1)):\n
Compute x-intercept of edge with horizontal line at y\n xinters = (x2 - x1) * (y -
y1) / (y2 - y1) + x1\n if x < xinters:\n inside = not inside\n \n
return inside\n``\n\n### Helper: `normalize_bbox_to_court`\n\n``python\ndef
normalize_bbox_to_court(bbox_pixels: Tuple[float, float, float, float],\n
frame_width: float, frame_height: float)\n) -> Tuple[float, float,
float, float]:\n \"\"\"Convert pixel bbox (x1, y1, x2, y2) to normalized [0, 1] range.\n
\n Args:\n bbox_pixels: (x1_px, y1_px, x2_px, y2_px) in pixels\n frame_width:
Width of video frame in pixels\n frame_height: Height of video frame in pixels\n \n
Returns:\n (norm_x1, norm_y1, norm_x2, norm_y2) where 0 ? coords ? 1\n \n
Scale-invariant: Yes (normalizes pixel coords).\n \n \"\"\"\n x1, y1, x2, y2 = bbox_pixels\n
if frame_width <= 0 or frame_height <= 0:\n return (0.0, 0.0, 0.0, 0.0)\n return (\n
max(0.0, min(1.0, x1 / frame_width)),\n max(0.0, min(1.0, y1 / frame_height)),\n
max(0.0, min(1.0, x2 / frame_width)),\n max(0.0, min(1.0, y2 / frame_height)),\n
)\n``\n\n### Compute `players_in_court` with court polygon:\n\n``python\ndef
compute_in_court_per_frame(\n detections_per_frame: List[List[Tuple[int, Tuple[float, float,
float, float]]],\n court_polygon: Optional[List[Tuple[float, float]]],\n frame_width:
float,\n frame_height: float,\n) -> List[int]:\n \"\"\"Count unique track IDs inside court
polygon per frame.\n \n Args:\n detections_per_frame: List of lists; index i is
frame i; inner list is \n [(track_id, bbox_pixels), ...].\n \n
court_polygon: Closed polygon (normalized 0?1) or None.\n frame_width, frame_height:
Video dimensions in pixels.\n \n Returns:\n List of counts; index i = unique in-
court track IDs in frame i.\n \n If no polygon: return counts of all detected persons (no
spatial filter).\n \n Edge cases:\n - polygon None ? count all detections per
frame\n - frame has no detections ? 0\n \n \"\"\"\n if court_polygon is None:\n
No spatial filter; count all persons\n return [len(frame_dets) for frame_dets in
detections_per_frame]\n \n in_court_counts = []\n for frame_dets in
detections_per_frame:\n in_court_ids = set()\n for track_id, bbox_px in
frame_dets:\n norm_bbox = normalize_bbox_to_court(bbox_px, frame_width,
frame_height)\n # Check if bbox center is in court\n norm_cx =
(norm_bbox[0] + norm_bbox[2]) / 2.0\n norm_cy = (norm_bbox[1] + norm_bbox[3]) / 2.0\n
if point_in_polygon((norm_cx, norm_cy), court_polygon):\n
in_court_ids.add(track_id)\n in_court_counts.append(len(in_court_ids))\n \n
return in_court_counts\n``\n\n--\n\n### Feature 2: `two_sided_motion`\n\n**Type:** Boolean / fraction |

```

```

Rally signal: rally = two-sided; one person feeding = one-sided\n\n``python\ndef
two_sided_motion(\n detections_per_frame: List[List[Tuple[int, Tuple[float, float, float,
float]]],\n net_line: Optional[Tuple[str, float]],\n court_polygon:
Optional[List[Tuple[float, float]]],\n frame_width: float,\n frame_height: float,\n) ->
float:\n \"\"\"Fraction of frames with moving (active) persons on BOTH net-halves.\n \n
Args:\n detections_per_frame: Per-frame detections (same as above).\n net_line:
Tuple (axis: 'x'|'y', net_coord: float) in normalized [0, 1].\n 'x' =
horizontal net (left/right halves); 'y' = vertical net.\n Example: ('y', 0.5) =
net at vertical midpoint.\n court_polygon: Court boundary (optional) ? only count in-
court persons.\n frame_width, frame_height: Video dimensions.\n \n Returns:\n
Fraction of frames (0.0?1.0) where ?1 moving person detected on each side\n of the net.
If no net_line or <2 frames, return 0.0.\n \n Requires velocity: This ALONE cannot
compute motion. It must be paired with\n per-frame active/moving persons (from
PersonDetector.extract_action_windows_from_chunk\n which flags frame_active_ids ?
velocity_thresh). Integrate that caller-side.\n \"\"\"
 if not net_line or not
detections_per_frame:\n return 0.0\n \n axis, net_coord = net_line\n if axis not
in ('x', 'y'):\n return 0.0\n \n two_sided_frames = 0\n total_frames = 0\n \n
for frame_dets in detections_per_frame:\n if not frame_dets:\n continue\n
\n total_frames += 1\n \n left_side = set()\n right_side = set()\n
\n for track_id, bbox_px in frame_dets:\n # Check court membership if polygon
given\n if court_polygon is not None:\n norm_bbox =
normalize_bbox_to_court(bbox_px, frame_width, frame_height)\n norm_cx =
(norm_bbox[0] + norm_bbox[2]) / 2.0\n norm_cy = (norm_bbox[1] + norm_bbox[3]) /
2.0\n if not point_in_polygon((norm_cx, norm_cy), court_polygon):\n
continue # Skip out-of-court\n \n # Normalize center to [0, 1]\n
norm_bbox = normalize_bbox_to_court(bbox_px, frame_width, frame_height)\n norm_cx =
(norm_bbox[0] + norm_bbox[2]) / 2.0\n norm_cy = (norm_bbox[1] + norm_bbox[3]) / 2.0\n
\n if axis == 'x':\n if norm_cx < net_coord:\n
left_side.add(track_id)\n else:\n right_side.add(track_id)\n
else: # axis == 'y'\n if norm_cy < net_coord:\n
left_side.add(track_id)\n else:\n right_side.add(track_id)\n
\n if left_side and right_side:\n two_sided_frames += 1\n \n
if total_frames == 0:\n return 0.0\n return float(two_sided_frames) /
float(total_frames)\n\n``\n\n**Caller integration note:** Feature `two_sided_motion` detects
spatial split; it does NOT filter for active motion. The caller must combine:\n1.
`detections_per_frame` = ALL detections in the window\n2. `active_ids_per_frame` =
List[Set[track_id]] (from PersonDetector Stage-1)\n3. Filter: only include frame_dets where
`track_id in active_ids_per_frame[frame_idx]`\n\n---\n\n## Feature 3:
`mean_player_motion`\n\n**Type:** Normalized velocity | **Rally signal:** rally = sustained;
resting/standing = low\n\n``python\ndef mean_player_motion(\n active_velocities:
List[float],\n) -> float:\n \"\"\"Aggregate normalized player velocity across active
frames.\n \n Args:\n active_velocities: List of per-frame peak velocities from
PersonDetector\n (already normalized by frame_width, units:
fraction/sec).\n Extracted directly from person_detector.py:\n
extract_action_windows_from_chunk() output \n
windows[i][\"mean_velocity\"] (averaged across active track IDs).\n \n Returns:\n
Mean velocity across frames. If empty, return 0.0.\n \n Scale-invariant: Yes (already
normalized by frame_width).\n \n Note: PersonDetector already computes this at window
level. For finer \n per-frame aggregation, sum normalized velocities and divide by frame
count.\n \"\"\"
 if not active_velocities:\n return 0.0\n return
sum(active_velocities) / len(active_velocities)\n\n---\n\n## Feature 4:
`in_court_ratio`\n\n**Type:** Fraction | **Rally signal:** track stability vs
flicker\n\n``python\ndef in_court_ratio(\n per_frame_in_court_counts: List[int],\n
min_players: int = 2,\n) -> float:\n \"\"\"Fraction of frames with ?min_players in court.\n
\n Args:\n per_frame_in_court_counts: List; index is frame; value is unique in-court
\n track count (from compute_in_court_per_frame).\n \n min_players: Minimum count to consider the frame \"occupied\" (default 2).\n \n Returns:\n
Fraction of frames meeting min_players threshold. If no frames, 0.0.\n \n Scale-invariant:
Yes (ratio, no pixels).\n \n Rationale: A stable rally has ?2 players in court most
frames. Flickering\n (detector missing one player for a frame or two) drops this ratio,
flagging\n noise vs. robust detection.\n \"\"\"
 if not per_frame_in_court_counts:\n
return 0.0\n occupied = sum(1 for count in per_frame_in_court_counts if count >=
min_players)\n return float(occupied) /
float(len(per_frame_in_court_counts))\n\n---\n\n## Feature 5:
`shuttle_player_colocation`\n\n**Type:** Fraction | **Rally signal:** held = high; in-flight =

```

```

mostly free space ? THE held-shuttle discriminator\n\n``python\ndef
shuttle_player_colocation(\n shuttle_points: List[Tuple[int, float, float, bool]], # (frame,
x, y, visible)\n detections_per_frame: List[List[Tuple[int, Tuple[float, float, float,
float]]],\n frame_width: float,\n frame_height: float,\n colocation_threshold: float =
0.3, # IOU threshold\n) -> float:\n \"\"\"Fraction of frames where visible shuttle
overlaps/adjacent to person box.\n \n Args:\n shuttle_points: List of (frame_idx,
x_px, y_px, visible).\n \n Returns:\n Empty if no shuttle trajectory available.\n \n detections_per_frame: Per-frame person detections (indexed by frame).\n \n frame_width,
frame_height: Video dimensions.\n \n colocation_threshold: IoU threshold (default 0.3 ?
\"touching or overlapping\").\n \n A held shuttle in hand ?0.7?1.0 IOU;
in-flight ?0.0?0.1.\n \n Returns:\n Fraction of frames where shuttle is visible AND
overlaps with a person box.\n \n If no shuttle points, return 0.0 (can't be held if not
visible).\n \n Scale-invariant: Yes (uses IOU, which is scale-invariant).\n \n
Rationale: A hand-held shuttle mostly stays inside/touching a person's bbox;\n a shuttlecock
in flight passes through the free space around them. High\n colocation (>0.5) is a strong
signal for a held / static shuttle, not a rally.\n \"\"\"
 if not shuttle_points:\n return 0.0\n \n from backend.utils.geometry import calculate_iou\n \n
colocated_frames = 0\n visible_frames = 0\n \n for frame_idx, shuttle_x, shuttle_y,
visible in shuttle_points:\n if not visible:\n continue\n \n
visible_frames += 1\n \n # Treat shuttle as tiny bbox ($\pm 2\%$ of frame width)\n
shuttle_radius_norm = 0.02 # 2% of frame width\n shuttle_radius_px = shuttle_radius_norm
* frame_width\n \n shuttle_bbox = (\n max(0, shuttle_x -
shuttle_radius_px),\n max(0, shuttle_y - shuttle_radius_px),\n
min(frame_width, shuttle_x + shuttle_radius_px),\n min(frame_height, shuttle_y +
shuttle_radius_px),\n)\n \n # Check overlap with any person in the frame\n
frame_dets = detections_per_frame[frame_idx] if frame_idx < len(detections_per_frame) else []\n
for _, person_bbox_px in frame_dets:\n iou = calculate_iou(shuttle_bbox,
person_bbox_px)\n if iou >= colocation_threshold:\n colocated_frames
+= 1\n break # One person overlapping is enough; move to next frame\n \n
if visible_frames == 0:\n return 0.0\n return float(colocated_frames) /
float(visible_frames)\n\n``\n\n---\n\n## Integration: Window-level aggregation\n\n**Called
from:** FusionSegmenter (P2) after court-mask filtering. Inputs are shuttle-seeded candidate
windows (from TrackNetRunner.trajectory_to_action_windows) plus PersonDetector Stage-1
output.\n\n``python\ndef compute_person_features(\n window_start: float,\n window_end:
float,\n detections_per_frame: List[List[Tuple[int, Tuple[float, float, float, float]]],\n
active_ids_per_frame: List[Set[int]], # From PersonDetector\n active_velocities_per_frame:
List[float], # From PersonDetector\n shuttle_trajectory: Optional[List[Tuple[int, float,
float, bool]]], # From TrackNetRunner\n fps: float,\n frame_width: float,\n
frame_height: float,\n court_polygon: Optional[List[Tuple[float, float]]] = None,\n
net_line: Optional[Tuple[str, float]] = None,\n) -> Dict[str, float]:\n \"\"\"Compute all
person-cue features for one candidate window.\n \n Returns dict with keys:\n \n -
players_in_court (float)\n - two_sided_motion (float, 0?1)\n - mean_player_motion
(float, normalized vel)\n - in_court_ratio (float, 0?1)\n \n shuttle_player_colocation
(float, 0?1)\n \"\"\"
 # Trim detections to window timeframe\n
window_frames = int((window_end - window_start) * fps)\n \n # Compute in-court counts per
frame (with optional polygon)\n in_court_counts = compute_in_court_per_frame(\n
detections_per_frame, court_polygon, frame_width, frame_height\n)\n \n # Filter
detections to only those with active motion\n active_dets_per_frame = []\n for frame_idx,
(dets, active_ids) in enumerate(zip(detections_per_frame, active_ids_per_frame)):\n
active_dets = [(tid, bbox) for tid, bbox in dets if tid in active_ids]\n
active_dets_per_frame.append(active_dets)\n \n return {\n \"players_in_court\":
players_in_court(in_court_counts),\n \"two_sided_motion\": two_sided_motion(\n
active_dets_per_frame, net_line, court_polygon, frame_width, frame_height\n),\n
 \"mean_player_motion\": mean_player_motion(active_velocities_per_frame),\n
 \"in_court_ratio\": in_court_ratio(in_court_counts, min_players=2),\n
 \"shuttle_player_colocation\": shuttle_player_colocation(\n shuttle_trajectory,
detections_per_frame, frame_width, frame_height\n),\n }\n\n``\n\n---\n\n## Edge Cases
& Defaults (Robust to Missing Inputs)\n\n| Condition | Behavior |\n|-----|-----|\n
| `court_polygon` is None | No spatial filtering; count all detected persons |\n| `net_line` is
None | `two_sided_motion` returns 0.0 (can't split without net) |\n| `shuttle_trajectory` is
None or empty | `shuttle_player_colocation` returns 0.0 (can't detect held shuttle) |\n| Window
has <2 detections | `players_in_court` ? 1; `two_sided_motion` = 0.0 |\n| All active frames have
0 velocity | `mean_player_motion` = 0.0 |\n| `frame_width` or `frame_height` ? 0 | Normalize
fails gracefully; return 0 / false for spatial tests |\n\n---\n\n## Persistence Format
(backend/infrastructure/database.py:240?256)\n\nThese features are stored in

```



```

`candidate_segments.stats` (JSON) alongside existing shuttle features:\n\n```\n\n{\n \"peak_velocity\": 0.35,\n \"mean_velocity\": 0.18,\n \"active_frame_count\": 42,\n \"track_id_count\": 2,\n \"players_in_court\": 2.0,\n \"two_sided_motion\": 0.92,\n \"mean_player_motion\": 0.21,\n \"in_court_ratio\": 0.95,\n \"shuttle_player_colocation\": 0.08\n}\n\nAdded to `distill_local.FEATURE_NAMES` (currently at\nbackend/eval/distill_local.py:40):\n\npython\nFEATURE_NAMES = [\n \"duration\",\n \"peak_velocity\",\n \"mean_velocity\",\n \"active_ratio\",\n \"net_crossings\",\n \"players_in_court\",\n \"two_sided_motion\",\n \"mean_player_motion\",\n \"in_court_ratio\",\n \"shuttle_player_colocation\"]\n\n---\n\n## Key Invariants (For Verification)\n\n1. **Scale/translation invariant:** No function uses raw pixel coordinates except internal normalization. All outputs are counts, ratios, or normalized velocities.\n\n2. **Camera-agnostic:** Court polygon and net line can be camera-specific; formulas don't assume court position/orientation.\n\n3. **Graceful degradation:** Missing shuttle/net/polygon ? zero or neutral (no crashes; feature extracts what's available).\n\n4. **Probabilistic:** All return floats in [0.0, 1.0] or non-negative reals, suitable for logistic regression classifier.\n\n5. **Testable in pure Python:** No GPU, no model weights ? synthetic trajectories + boxes can test all paths.\n\n--\n\n## References (File:Line)\n\nPersonDetector.extract_action_windows_from_chunk: person_detector.py:138?254\n\nTrackNetRunner.parse_trajectory_csv: tracknet_runner.py:232?249\n\nrally_gate.estimate_play_axis: rally_gate.py:55?66\n\n- distill_local.FEATURE_NAMES: distill_local.py:40\n\n- database.add_candidate_segment: database.py:240?261\n\n- geometry.normalize_bbox: geometry.py (model for normalization)\n\n- \"anchors\": [\"backend/pipeline/detectors/person_detector.py:98?136\", \"backend/pipeline/detectors/tracknet_runner.py:80?84\", \"backend/pipeline/detectors/tracknet_runner.py:232?249\", \"backend/eval/distill_local.py:40\", \"backend/infrastructure/database.py:240?261\", \"backend/utis/geometry.py:23?40\", \"docs/MULTI_SIGNAL_FUSION_PLAN.md:§3.2?§4\"], \"risks\": [\"shuttle_player_colocation relies on ShuttlePoint.visible flag? if TrackNet never provides visible=False frames, colocation will be computed only on detected frames, potentially biasing toward in-flight (low colocation). Validate that trajectory CSV contains visibility column correctly.\"], \"two_sided_motion without PersonDetector active_ids filter will include stationary persons, conflating spatial occupancy with motion. Caller must pair with active_ids_per_frame from Stage-1.\"], \"normalize_bbox_to_court silently clamps coords to [0, 1]; out-of-bounds bboxes (possible if detector errs) become edge-pinned. Court polygon tests should handle this; document assumption.\"], \"point_in_polygon ray-casting assumes polygon is simple (non-self-intersecting); complex/malformed court polygons from manual annotation may produce incorrect membership. Add polygon-validation in config loading (P2 task).\"], {\"findings\": \"### NO-REGRESSION RISKS FOR FusionSegmenter + Gate A WIRING\n\n**The FusionSegmenter is NEW (not yet created). It will wrap shuttle-seeded candidates with person-cue gating and person features, then apply Gate A (min_crossings gate) that already exists in rally_gate.py but is **NOT wired into production**. The regression surface is precisely: wasb_hybrid and tracknet_hybrid must output IDENTICAL byte-for-byte results when min_crossings=0 (no gate) and fusion is disabled (cue_flags all false).**\n\n### Core Regression Invariants (MUST NOT BREAK)\n\n1. Trajectory hybrid output immutability (the hard contract)\n\n- `wasb_hybrid.process_video()` must return identical `List[Dict[str, Any]]` regardless of FusionSegmenter's existence or config values\n\n- `tracknet_hybrid.process_video()` must return identical output (both subclass TrajectoryHybridSegmenter;\n\ntest_trajectory_hybrid_equivalence.py::test_detector_identity_flows_into_db_rows line:60?68 already proves they diverge only by family/name)\n\n- The fusion wiring CANNOT change\n\n`trajectory_hybrid.py::process_video` ? it is only a **consumer** of the candidate windows trajectory produces, not a **modifier** of the trajectory logic\n\n- **Risk anchor:** Any code path that touches `TrajectoryHybridSegmenter.process_video()` (lines 81?262) must remain locked to the existing logic; fusion runs only AFTER Phase 2 candidates are finalized\n\n2. Candidate window enumeration byte-identity\n\n- `runner.trajectory_to_action_windows()` (called line:161?165 in trajectory_hybrid.py) signature and output MUST NOT change\n\n- The windows dict schema (start, end, active_frame_count, peak_velocity, mean_velocity, track_id_count, chunk_index) is immutable ? no new keys, no reordered keys\n\n- **Risk:** A fusion segmenter that pre-filters windows during the Phase 2 call will break this. DESIGN: fusion runs POST-Phase-2, consuming the full candidate list, not replacing\n\n`trajectory_to_action_windows`\n\n3. Database candidate persistence unchanged\n\n- `add_candidate_segment()` call (lines 185?190) must persist the SAME stats dict and detection_params dict as before\n\n- detection_params must contain ONLY the keys that trajectory_hybrid.py currently sets: detector, velocity_thresh, merge_gap, min_action_window_duration, + family-specific runner_params (weights_path, etc.)\n\n- **Risk:** FusionSegmenter must NOT add new keys to detection_params when calling add_candidate_segment ? that would change the candidate table schema and break LOVO training reproducibility\n\n- **Design:** FusionSegmenter persists person features into a SEPARATE stats dict at a new key

```

(e.g., `"fusion_stats"`) or as a new optional column on `candidate_segments`, not mixed into the trajectory stats\n\n\*\*4. AI handoff window identity\*\*\n - Phase 3 (lines 195?236) handoff window boundaries must not shift; padded clip extraction, provider call, timestamp adjustment all fixed\n - \*\*Risk:\*\* If fusion gates windows before Phase 3, the provider will analyze a DIFFERENT set of clips (fewer / different time ranges)\n\n\*\*5. Shot-count and metadata identity\*\*\n - `_shot_count_from()` (lines 67?79) is a pure function with fixed logic; output must not change for the same input segment\n - Metadata dict (lines 245?249) keys must not change; detector tag is `f"{family}-hybrid-{model_name}"`, immutable by design\n\n### Specific Code Risks (with line anchors)\n\n\*\*Risk A: min\_crossings in IndexingConfig\*\*\n - FusionSegmenter needs to read `min_crossings` from config, but this knob does NOT exist in `backend/config/models.py`\n - \*\*File:Line:\*\* `IndexingConfig` model (`models.py:78?127`) must ADD a new field `min_crossings: int = 0` as default\n - \*\*Impact:\*\* If added with default=2 (instead of default=0), every existing run changes behavior (gate becomes active) ? regression\n - \*\*Fix:\*\* Default must be 0 (gate disabled). The variant in `experiment.py:Variant.min_crossings` (line:108) already defaults to 0 ? match that\n\n\*\*Risk B: Calling rally\_gate.gate\_windows() in a new code path\*\*\n - FusionSegmenter will need to import and call `rally_gate.gate_windows(points, windows, fps, min_crossings)` on the trajectory points\n - Current callers: `eval drivers only` (`distill_local.py:56`, `export_wasb_segments.py`, `calibrate_local.py`) ? NOT production segmenters\n - \*\*Impact:\*\* If FusionSegmenter calls `gate_windows` with `min_crossings=0` (disabled), it must return identical `GatedWindow` objects to what `trajectory_to_action_windows` returns\n - \*\*Design risk:\*\* `gate_windows` modifies window tuples (adds crossings, visible\_points, kept fields) ? the base windows are unchanged, only the result type changes\n - \*\*Mitigation:\*\* FusionSegmenter reads `gated.crossings` for stats/logging but does NOT filter windows when `min_crossings=0`\n\n\*\*Risk C: Person-detector integration in FusionSegmenter\*\*\n - FusionSegmenter will instantiate `PersonDetector` (from `backend/pipeline/detectors/person_detector.py`) and call `extract_action_windows_from_chunk()` over shuttle-seeded windows + padding\n - \*\*Code anchor:\*\* `PersonDetector.__init__(config)` (line:56) reads `indexing.{use_gpu, detector_model, detector_score_threshold, min_action_window_duration}`\n - \*\*Risk:\*\* If FusionSegmenter modifies the config dict passed to `PersonDetector`, or if `PersonDetector`'s `lazy_load_model()` throws, the entire segmenter fails (not just fusion)\n - \*\*Impact:\*\* With fusion disabled (`cue_flags={"person": False}`), `PersonDetector` must NOT be instantiated or must be a no-op\n - \*\*Design:\*\* FusionSegmenter checks `cue_flags["person"]` at runtime; if false, skip `PersonDetector` entirely and return unmodified candidates\n\n\*\*Risk D: Candidate stats mutation\*\*\n - Current trajectory code never modifies the windows dict after creation (lines 161?165); each window is read once to compute stats (lines 179?190) and never touched again\n - \*\*Risk:\*\* If FusionSegmenter mutates `all_candidate_windows` (e.g., adds `"fusion_stats"` keys), those mutations persist into Phase 3 handoff and Phase 4 DB storage\n - \*\*Impact:\*\* The provider sees modified window dicts instead of the canonical ones ? timing/metadata may diverge\n - \*\*Design:\*\* FusionSegmenter must work with COPIES of candidate windows or use a separate data structure (e.g., a parallel list of fusion\_stats dicts, keyed by window index)\n\n\*\*Risk E: player-count gate (Gate B) not yet wired\*\*\n - `Variant.min_players` is already in `experiment.py` (line:109) but persisted person features (`players_in_court`) do not exist yet\n - \*\*Risk:\*\* A FusionSegmenter that gates on `min_players > 0` before the feature is persisted will silently become a no-op (`test_disabled_cues_byte_identical_to_control` will pass but for the wrong reason)\n - \*\*Mitigation:\*\* `min_players` must default to 0 in both `IndexingConfig` and `Variant`; feature persistence is P3, not P2\n\n--\n\n### PURE-LOGIC TEST PLAN FOR DEFAULT-OFF WIRING\n\n\*\*All tests are UNIT, OFFLINE, with ZERO GPU/torch/cv2 dependencies.\*\* They prove the logic gates and feature assembly without running real video. Synthetic fixtures are key.\n\n### Test Suite 1: Config + Variant + Equivalence Harness (no new tests needed ? reuse existing)\n\n\*\*Reuse from `test_config.py`:test\_load\_default\_config (line:18): verify `min_crossings` defaults to 0 (when added to `IndexingConfig`)\n - `test_file_override` (line:28): verify `config.json` can override `min_crossings`, `min_players` without breaking other settings\n - `test_extra_keys_tolerated` (line:51): `min_crossings` in old configs must not break loading\n\n\*\*Reuse from `test_experiment.py`:test\_disabled\_cues\_byte\_identical\_to\_control (line:78): \*\*THE KEY REGRESSION TEST\*\*\n - `disabled = Variant(name="fusion_off", cue_flags={"person": False, "audio": False}, min_crossings=0, min_players=0)`\n - `assert predict(disabled, vc) == predict(CONTROL, vc)` ? must still pass after FusionSegmenter exists\n - `test_gate_a_drops_low_crossing_window` (line:94): already tests the gate logic in pure Python\n - `vc = _held_shuttle_video()` (line:30) has intervals with `net_crossings=[4.0, 3.0, 0.0]`\n - `gate_a = Variant(name="gateA", min_crossings=2)`\n - `preds = predict(gate_a, vc)`; `assert (12.0, 13.0) not in preds` (line:98) ? the held-shuttle window is correctly dropped\n\n\*\*New: `test_fusion_default_off_preserves_trajectory_output`\*\*\n - \*\*Purpose:\*\* Prove that enabling FusionSegmenter in config but setting `cue_flags` all false does not modify trajectory output\n - \*\*Fixture:\*\* Mock a minimal config: `{ "indexing":`

```

{"default_segmenter\\": "\\fusion\\", "\\min_crossings\\": 0, "\\min_players\\": 0, ...}}\\n -
Setup: Create a trajectory.csv with 10 points (synthetic, 5 visible); run
TrackNetRunner.trajectory_to_action_windows() to get baseline windows W\\n - **Action**: Create
FusionSegmenter(db_mock, config), call an internal method that applies Gate A with
min_crossings=0\\n - **Assert**: The returned windows are W unchanged (same interval tuples,
same stats dicts)\\n\\n### Test Suite 2: Rally Gate Logic in Isolation (extend
test_rally_gate.py)\\n\\n**Reuse existing (already comprehensive)**\\n -
test_estimate_play_axis_picks_larger_spread (line:10): axis detection\\n -
test_count_net_crossings_counts_sign_changes (line:17): the crossing counter\\n -
test_deadband_ignores_jitter_near_net (line:22): jitter suppression\\n -
test_invisible_points_are_skipped (line:27): visibility filter\\n -
test_gate_kills_single_toss_keeps_multi_crossing_rally (line:32): **the held-shuttle use
case**\\n - Fixture: rally=[10 frames, alternating top/bottom] + toss=[2 frames, one
crossing]\\n - windows=[(0.0, 1.0) rally_window, (2.0, 3.0) toss_window]\\n - gated =
gate_windows(pts, windows, fps=10, min_crossings=2)\\n - Assert gated[0].kept=True,
gated[1].kept=False ? toss is dropped\\n - Assert apply_gate() returns only [(0.0,
1.0)]\\n\\n**New: test_gate_with_min_crossings_0_is_no_op**\\n - **Purpose**: min_crossings=0
must NOT filter any window (all windows pass)\\n - **Fixture**: P(0, 0, 10), P(1, 0, 200) (one
crossing); windows=[(0.0, 0.5)]\\n - **Action**: gated = gate_windows(pts, windows, fps=30,
min_crossings=0)\\n - **Assert**: gated[0].kept=True (even though crossings=1; 1 >= 0)\\n -
Assert: apply_gate(..., min_crossings=0) returns the window unchanged\\n\\n**New:
test_crossing_count_stability_across_fps**\\n - **Purpose**: net_crossings feature is fps-
invariant (only frame indices matter)\\n - **Fixture**: Same trajectory, different fps values
(10, 30, 60)\\n - **Action**: gated_10 = gate_windows(pts, windows, fps=10, ...); gated_30 =
...; gated_60 = ...\\n - **Assert**: All return the same crossing count (the distance in
coordinate space is time-agnostic)\\n\\n### Test Suite 3: PersonDetector + Window Enrichment (new,
reuse test_person_detector.py idiom)\\n\\n**Reuse existing (already in
test_person_detector.py)**\\n - test_extract_action_windows_returns_enriched_dicts (line:37):
windows have the required keys\\n - test_detect_and_track_filters_persons_and_returns_track_ids
(line:60): COCO person==1 filter + tracker\\n\\n**New:
test_person_windows_with_cue_disabled_not_computed**\\n - **Purpose**: If
cue_flags[\\\"person\\\"]=False, PersonDetector is never instantiated\\n - **Fixture**:
FusionSegmenter with cue_flags={\\\"person\\\": False}\\n - **Action**: Call
FusionSegmenter.process_video() (or a test-facing _fusion_gate method)\\n - **Assert**:
PersonDetector is never created (check __init__ is not called), no CV2 VideoCapture mock
needed\\n\\n**New: test_person_detector_returns_window_dict_with_motion_stats**\\n - **Purpose**:
PersonDetector.extract_action_windows_from_chunk() returns dicts with motion stats\\n -
Fixture: _make_cap_mock(num_frames=300), _make_detections(num_frames=300, step=5,
track_id=1) (motion is clear)\\n - **Setup**: pd = PersonDetector({\\\"indexing\\\": {...}}); mock
the model + tracker\\n - **Action**: windows =
pd.extract_action_windows_from_chunk(\\\"chunk.mp4\\\", chunk_start=0, velocity_thresh=0.05,
merge_gap=2.0, frame_skip=1, chunk_index=0)\\n - **Assert**: len(windows) >= 1; w = windows[0];
assert {\\\"start\\\", \\\"end\\\", \\\"active_frame_count\\\", \\\"peak_velocity\\\", \\\"mean_velocity\\\",
\\\"track_id_count\\\", \\\"chunk_index\\\"} <= set(w.keys())\\n - **Assert**: w[\\\"start\\\"] >=
chunk_start, w[\\\"active_frame_count\\\"] > 0, w[\\\"peak_velocity\\\"] >= w[\\\"mean_velocity\\\"] > 0,
w[\\\"track_id_count\\\"] >= 1\\n\\n### Test Suite 4: Fusion Gate Logic (Point-in-Polygon, Player
Count, AND Composition)\\n\\n**New: test_court_mask_polygon_filters_persons_outside_court**\\n -
Purpose: A court-region polygon (4 corners) filters persons outside it\\n - **Fixture**:
Synthetic person boxes: (100, 100, 150, 150) inside court, (10, 10, 30, 30) outside court;
court_mask = polygon with corners (50, 50), (250, 50), (250, 250), (50, 250)\\n - **Setup**:
Implement a simple point-in-polygon filter (ray casting or cross product)\\n - **Action**:
filtered = [box for box in boxes if court_mask.contains(box_center(box))]\\n - **Assert**:
Inside box kept, outside box dropped\\n\\n**New: test_player_count_constraint_expects_2_or_4**\\n -
Purpose: Player-count gate (Gate B) expects 2 (singles) or 4 (doubles); other counts are
transient/noise\\n - **Fixture**: A window with per-frame track counts [1, 1, 2, 2, 2, 1, 1]
(median=2); another with [0, 1, 1, 0] (median=1, too low for singles)\\n - **Setup**: Extract
median track count per window\\n - **Action**: passed = [w for w in windows if
median_track_count(w) in (2, 4)]\\n - **Assert**: First window passes (median=2), second fails
(median=1 < 2)\\n\\n**New: test_gate_a_and_gate_b_composition_both_must_pass**\\n - **Purpose**:
A window is kept iff min_crossings AND min_players gates both pass (AND gate)\\n - **Fixture**:
VideoCandidates with 4 windows:\\n - W1: net_crossings=3, players_in_court=2 ? should PASS
(both gates true)\\n - W2: net_crossings=1, players_in_court=2 ? should FAIL (crossings <
2)\\n - W3: net_crossings=3, players_in_court=1 ? should FAIL (players < 2)\\n - W4:
net_crossings=1, players_in_court=1 ? should FAIL (both gates false)\\n - **Action**:
variant_gate_a_b = Variant(min_crossings=2, min_players=2); results = predict(variant_gate_a_b,

```

```

vc)\n - **Assert**: results include only W1's interval; W2, W3, W4 are excluded\n\n**New:
test_default_off_gates_all_inclusive (the byte-identity test)\n - **Purpose**:
min_crossings=0 AND min_players=0 means all windows pass (control behavior)\n - **Fixture**:
Same 4-window vc from above (mix of passing/failing scenarios)\n - **Action**: variant_off =
Variant(min_crossings=0, min_players=0); results = predict(variant_off, vc)\n - **Assert**:
results == predict(CONTROL, vc); all 4 window intervals are present (merged by merge_gap)\n\n###
Test Suite 5: Synthetic Trajectories + Persons (Composite Fixture)\n\n**New:
test_shuttle_player_colocation_feature_empty_when_person_cue_off\n - **Purpose**: When
cue_flags["person"]=False, no shuttle_player_colocation feature is computed\n - **Fixture**:
A candidate_segment row with net_crossings=4 but NO shuttle_player_colocation key in stats\n -
Action: FusionSegmenter.predict(...) with cue_flags={"person": False}\n - **Assert**:
Feature vector includes net_crossings but NOT shuttle_player_colocation; classifier.predict()
does not fail (feature list is static, controlled by variant.feature_names)\n\n**New:
test_two_sided_motion_detected_correctly\n - **Purpose**: A rally has players on both net-
halves; a feed has players only on one half\n - **Fixture**: Synthetic person tracks:
rally_tracks = [(0, frame, 100, 100), (0, frame, 900, 100)] (IDs 1,2 on opposite sides);
feed_tracks = [(0, frame, 100, 100), (0, frame, 150, 100)] (both IDs on same side)\n -
Setup: Compute net_half membership (left=x < net_x, right=x >= net_x) for each track; check
if both halves are represented in a window\n - **Action**: two_sided =
count_sides(rally_tracks, net_x=500) > 1; count_sides(feed_tracks, net_x=500) == 1\n -
Assert: two_sided for rally=True, feed=False\n\n**New:
test_in_court_ratio_tracks_occupancy_stability\n - **Purpose**: in_court_ratio = (frames
with ?2 in-court players) / (total frames in window)\n - **Fixture**: A 10-frame window:
frames 0?2 have 0 players, frames 3?9 have 2 players\n - **Action**: ratio = sum(1 for f in
frames if track_count(f) >= 2) / len(frames)\n - **Assert**: ratio = 7/10 = 0.7\n\n### Test
Suite 6: Byte-Identical Assertions (No-Regression Guard)\n\n**New:
test_fusion_segmenter_with_all_cues_disabled_matches_trajectory_hybrid\n - **Purpose**:
FusionSegmenter instantiated with fusion disabled must return the exact same rows as
trajectory_hybrid\n - **Fixture**: Mocked video_id, video_path, provider, db; trajectory CSV
with 10 points; trajectoryto_action_windows returns 3 candidate windows\n - **Setup**: \n
- trajectory_hybrid_res = TrackNetHybridSegmenter(db, {"indexing": {...},
...}).process_video(...)\n - fusion_res = FusionSegmenter(db, {"indexing": {...},
"min_crossings": 0, "min_players": 0, cue_flags: {"person": False, ...}},
...).process_video(...)\n - **Assert**: len(fusion_res) == len(trajectory_hybrid_res); for
each seg in both: seg["start_time"], seg["end_time"], seg["metadata"]["detector"],
seg["metadata"]["shot_count"] are identical\n\n**New:
test_persisted_candidate_stats_unchanged_with_fusion_off\n - **Purpose**: The stats dict
added to candidate_segments must NOT change when fusion is wired\n - **Fixture**: Mocked db
with add_candidate_segment spy\n - **Setup**: FusionSegmenter(...).process_video(...) with
min_crossings=0, cue_flags all false\n - **Action**: Capture the stats dict passed to
add_candidate_segment()\n - **Assert**: stats.keys() == {peak_velocity, mean_velocity,
active_frame_count, track_id_count}; no "net_crossings" key (crossing count is a feature, not
a stat)\n\n**New: test_handoff_window_boundaries_stable_with_min_crossings_0\n -
Purpose: Window start/end times for Phase 3 AI handoff must not shift when Gate A is
disabled\n - **Fixture**: Candidate windows from trajectory; min_crossings=0 configured\n -
Setup: Trace process_candidate_window() calls (line:201?224 in trajectory_hybrid.py)\n -
Action: Extract (w_start, w_end) for each window\n - **Assert**: (w_start, w_end) ==
(window["start"] - handoff_padding, window["end"] + handoff_padding) for all windows (no
filtering)\n\n--\n\n### SYNTHETIC FIXTURES FOR PURE-LOGIC PROOF\n\n### Fixture 1:
VideoCandidates for Held-Shuttle vs Rally (from
test_experiment.py::held_shuttle_video)\n\n``python\n# The canonical held-shuttle false
fire\nheld_shuttle_video(name="held_shuttle") = VideoCandidates(\n
name="held_shuttle",\n intervals=[(0.0, 10.0), (20.0, 30.0), (12.0, 13.0)], # third one is
the held-shuttle\n features={"net_crossings": [4.0, 3.0, 0.0]}, # third has zero
crossings\n gts=[(0.0, 10.0), (20.0, 30.0)], # ground truth excludes the held-
shuttle\n)\n\n``\n\n**Use case**: test_gate_a_drops_low_crossing_window,
test_default_off_gates_all_inclusive\n\n### Fixture 2: Trajectory Points (from
test_rally_gate.py::P and test_yolo_hybrid.py::make_detections)\n\n``python\ndef P(frame, x,
y, visible=True):\n return TrajectoryPoint(frame=frame, visible=visible, x=x, y=y)\n\n#
Rally: shuttle crosses net multiple times\nrally_pts = [\n P(0, 500, 50), P(1, 500, 200),
P(2, 500, 50), P(3, 500, 200), # alternates (3+ crossings)\n]\n\n# Held-shuttle: one crossing,
then static\nheld_pts = [\n P(0, 500, 50), P(1, 500, 250), # one crossing\n P(2, 500,
250), P(3, 500, 250), # held at top\n]\n\n# Toss/feed: one crossing only\ntoss_pts = [P(0, 500,
50), P(1, 500, 250)] # one crossing, no return\n\n``\n\n**Use case**:
test_gate_kills_single_toss_keeps_multi_crossing_rally,

```

```

test_crossing_count_stability_across_fps\n\n### Fixture 3: Person Detections (from
test_person_detector.py::_make_detections)\n\n`python\ndef _make_detections(num_frames=300,
step=5, track_id=1):\n \"\"\"One steadily-moving person per frame; box advances `step`
px/frame.\"\"\"\n return [[[track_id, (float(i * step), 0.0, float(i * step + 10), 10.0)]]\nfor i in range(num_frames)]\n\n# Rally scenario: two players, opposite sides of net (net at
x=500)\nrally_persons = [\n # Frame 0: player 1 left, player 2 right\n [(1, (100, 100,
150, 200)), (2, (900, 100, 950, 200))],\n # Frame 1: both move (motion > 0.05 normalized)\n [(1, (105, 100, 155, 200)), (2, (895, 100, 945, 200))],\n] * 150 # 300 frames total\n\n# Feed
scenario: one player only\nfeed_persons = [\n # Frames 0?149: no motion\n [(1, (100, 100,
150, 200))]] * 150,\n # Frames 150?299: motion but only one side\n [(1, (105, 100, 155,
200))]] * 150,\n]\n\n`python\n#
Court Polygon\n\n`python\n#
Court region (normalized coordinates, or pixel coords after calibration)\ncourt_mask =
Polygon(corners=[(50, 50), (250, 50), (250, 250), (50, 250)])\n\n# Person box inside
court\ninside_box = (100, 100, 150, 200)\n\n# Person box outside court (spectator or adjacent
court)\noutside_box = (10, 10, 30, 30)\n\n`python\n#
Use case*:
test_two_sided_motion_detected_correctly,
test_player_count_constraint_expects_2_or_4\n\n### Fixture 4: Court Polygon\n\n`python\n#
Court region (normalized coordinates, or pixel coords after calibration)\ncourt_mask =
Polygon(corners=[(50, 50), (250, 50), (250, 250), (50, 250)])\n\n# Person box inside
court\ninside_box = (100, 100, 150, 200)\n\n# Person box outside court (spectator or adjacent
court)\noutside_box = (10, 10, 30, 30)\n\n`python\n#
Use case*:
test_court_mask_polygon_filters_persons_outside_court\n\n### Fixture 5: Mixed Window Features
(for composite AND-gate test)\n\n`python\n# 4-window fixture combining crossing count + player
count\nvc = VideoCandidates(\n name=\"mixed\", \n intervals=[(0.0, 10.0), (20.0, 30.0),
(40.0, 50.0), (60.0, 70.0)], \n features={\n \"net_crossings\": [3.0, 1.0, 3.0, 1.0], \n
\"players_in_court\": [2.0, 2.0, 1.0, 1.0], \n }, \n gts=[(0.0, 10.0), (20.0, 30.0)], #
golden labels define real rallies\n)\n\n# With min_crossings=2, min_players=2:\n# W1:
crossings=3, players=2 ? PASS (both gates)\n# W2: crossings=1, players=2 ? FAIL (crossings <
2)\n# W3: crossings=3, players=1 ? FAIL (players < 2)\n# W4: crossings=1, players=1 ? FAIL (both
gates)\n# Expected: only W1 in results\n\n`python\n#
Use case*:
test_gate_a_and_gate_b_composition_both_must_pass\n\n--\n\n### OWNER-VERIFIED GPU TEST CASES
(REAL VIDEO, NOT PURE LOGIC)\n\n`python\n#
These CANNOT run in the dev container (no GPU, no WSL). Owner
must run on a GPU machine with real badminton footage.*\n\n### GPU Test 1: Held-Shuttle False
Positive Fix (the Original Bug Report)\n\n`python\n#
Owner observation*: A player holds/flicks the
shuttle between points, and the detector fires a \"rally.\"*\n\n`python\n#
Setup*: A static-tripod video
clip (?30s) with:\n - A real rally in frames 0?300 (shuttle crosses net multiple times)\n -
A held-shuttle / between-points feed in frames 300?330 (player holds shuttle, single toss)\n -
Post-feed dead time in frames 330?900\n\n`python\n#
Run wasb_hybrid (or tracknet_hybrid) baseline (Gate A
disabled, min_crossings=0):*\n - Expected: Rally detected in frames 0?300; held-shuttle false
fire in frames 300?330\n - **Verify*: Two play segments returned\n\n`python\n#
Run FusionSegmenter
with Gate A enabled (min_crossings=2):*\n - Expected: Rally detected in frames 0?300; held-
shuttle false fire DROPPED (crossings < 2)\n - **Verify*: One play segment returned (the
rally only)\n - **Quantify*: Held-shuttle window has net_crossings=1 (fixture: person tosses
once, no return)\n\n### GPU Test 2: Gate B Player-Count Rejection (Future, but prototype
structure)\n\n`python\n#
Setup*: A static-tripod clip with:\n - A real rally (frames 0?300): both
players visible, moving on opposite sides\n - A warmup knock-up (frames 320?350): ONE player
flicking the shuttle solo\n - Another real rally (frames 400?500): both players, full rally
motion\n\n`python\n#
Run baseline (person cue disabled, only Gate A):*\n - Expected: Both real rallies
detected, warmup may be detected if crossing count is high\n\n`python\n#
Run FusionSegmenter with Gate B
enabled (min_players=2):*\n - Expected: Solo warmup is dropped (only 1 player visible); both
real rallies kept (?2 players)\n - **Verify*: players_in_court feature for warmup window is
1, for rallies is 2\n\n### GPU Test 3: Two-Sided Motion Discrimination\n\n`python\n#
Setup*: A static-
tripod clip with:\n - A real rally (frames 0?300): players alternate sides (motion on both
left and right of net)\n - A service-return feed (frames 320?350): server tosses, receiver
feeds back (one-sided motion on server side only)\n\n`python\n#
Run FusionSegmenter with learned
classifier (if P3 lands):*\n - Expected: Rally has two_sided_motion=True; feed has
two_sided_motion=False\n - **Verify*: two_sided_motion feature correctly labeled; classifier
down-weights feed (high shuttle-colocation, one-sided)\n\n### GPU Test 4: Byte-Identity
Regression on Full Video\n\n`python\n#
Setup*: A 10-minute static-tripod match video processed with the
owner's normal config.\n\n`python\n#
Run wasb_hybrid.process_video() ? output_baseline.json (segments,
metadata):*\n - Record: segment count, start/end times, detector field, shot_count\n\n`python\n#
Run
FusionSegmenter with fusion disabled (cue_flags all False, min_crossings=0, min_players=0) ?
output_fusion_off.json:*\n - Expected: output_baseline.json == output_fusion_off.json (byte-
identical segment count, times, metadata)\n - **Verify*: `python -m pytest -xvs
tests/test_trajectory_hybrid_equivalence.py::test_detector_identity_flows_into_db_rows` still
passes\n\n### GPU Test 5: Performance (fusion overhead)\n\n`python\n#
Setup*: A 30-minute static-tripod
clip.\n\n`python\n#
Baseline*: wasb_hybrid runtime (GPUs WSL trajectory + Gemini handoff)\n\n`python\n#
With Gate
A (no person cue):*\n - FusionSegmenter with min_crossings=2, cue_flags[\"person\"]=False\n -
Expected: Negligible overhead (pure Python gate logic on trajectory points)\n - **Measure*:
Total runtime, Phase 2 runtime (no person detection, no extra CV2 calls)\n\n`python\n#
With Gate A +

```

person cue (P3+)\*\*: FusionSegmenter with min\_crossings=2, min\_players=2, cue\_flags["person"]=True\n - Expected: Additional cost = torchvision detector over shuttle-seeded windows + padding (frame-skip=3 by default)\n - \*\*Measure\*\*: Relative cost vs baseline; Phase 1?2 should be <10% additional wall time (person is fast; shuttle runs full-video in WSL in parallel)\n\n--\n\n## SUMMARY: WHAT MUST BE OWNER-VERIFIED\n\n1. \*\*Binary regression gate (pure offline)\*\*: `test\_disabled\_cues\_byte\_identical\_to\_control` still passes after FusionSegmenter lands\n2. \*\*held-shuttle-specific false-positive fix\*\*: Gate A (min\_crossings=2) on real video drops the single-toss service-feed window\n3. \*\*no production pathway corruption\*\*: Phase 3 AI handoff uses the correct (unfiltered) window boundaries when Gate A is disabled\n4. \*\*config schema safety\*\*: IndexingConfig.min\_crossings defaults to 0, Pydantic validation does not break on old config.json files\n5. \*\*byte-identity on full video\*\*: wasb\_hybrid output == FusionSegmenter(min\_crossings=0, ...) output on a representative 10-minute clip\n6. \*\*person cue no-op proof\*\*: PersonDetector is never instantiated when cue\_flags["person"]=False (no CV2 overhead)\n\n", "anchors": ["backend/pipeline/detectors/rally\_gate.py:1?20 (Gate A docstring and pure functions)", "backend/pipeline/segmenters/trajectory\_hybrid.py:81?262 (process\_video pipeline, immutable contract)", "backend/pipeline/segmenters/trajectory\_hybrid.py:161?165 (trajectory\_to\_action\_windows call, candidate window enumeration)", "backend/pipeline/segmenters/trajectory\_hybrid.py:185?190 (add\_candidate\_segment call, stats persistence)", "backend/pipeline/segmenters/trajectory\_hybrid.py:245?249 (metadata dict assembly)", "backend/config/models.py:78?127 (IndexingConfig, where min\_crossings will live)", "backend/pipeline/detectors/person\_detector.py:48?254 (PersonDetector full contract)", "backend/pipeline/detectors/person\_detector.py:138?254 (extract\_action\_windows\_from\_chunk, motion-window assembly)", "backend/eval/distill\_local.py:40 (FEATURE\_NAMES = ['duration', 'peak\_velocity', 'mean\_velocity', 'active\_ratio', 'net\_crossings'])", "backend/eval/distill\_local.py:56 (gate\_windows call with min\_crossings=0 in build\_candidates)", "backend/eval/experiment.py:73?102 (Variant dataclass definition, min\_crossings and min\_players fields)", "backend/eval/experiment.py:148 (CONTROL = Variant(name='control'), the pinned baseline)", "tests/test\_experiment.py:78?99 (test\_disabled\_cues\_byte\_identical\_to\_control and test\_gate\_a\_drops\_low\_crossing\_window)", "tests/test\_rally\_gate.py:32?41 (test\_gate\_kills\_single\_toss\_keeps\_multi\_crossing\_rally, the held-shuttle use case)", "tests/test\_person\_detector.py:37?57 (test\_extract\_action\_windows\_returns\_enriched\_dicts, motion stat assembly)", "tests/test\_trajectory\_hybrid\_equivalence.py:60?68 (test\_detector\_identity\_flows\_into\_db\_rows, twin equivalence)", "tests/test\_config.py:18?49 (test\_load\_default\_config and test\_file\_override, config safety)"], "risks": ["IndexingConfig.min\_crossings added with wrong default (must be 0, not 2) breaks existing videos", "FusionSegmenter calls rally\_gate.gate\_windows() with min\_crossings > 0 before all features are persisted causes train/inference mismatch", "PersonDetector instantiated unconditionally even when cue\_flags['person']=False adds unwanted GPU/torch dependency", "FusionSegmenter mutates candidate\_windows dicts in-place corrupts Phase 3 provider handoff window boundaries", "Court-mask polygon filtering applied during Phase 2 instead of post-Phase-2 pre-filter changes window enumeration", "min\_players gate references non-existent 'players\_in\_court' feature before P3 persistence lands, silently no-ops", "config.json override of min\_crossings to a high value (e.g., 10) breaks existing production videos", "Regression: wasb\_hybrid and tracknet\_hybrid outputs diverge when FusionSegmenter logic is added to base TrajectoryHybridSegmenter", "net\_crossings feature persisted as stats key instead of separate feature breaks distill\_local.py FEATURE\_NAMES contract", "FusionSegmenter default segmenter string in registry causes yolo\_hybrid vs fusion selection ambiguity"]]]].

Hard constraints (non-negotiable):

- Shuttle stays PRIMARY (windows seed from the WASB/TrackNet trajectory); the person cue only ENRICHES + ADJUDICATES shuttle-seeded windows, never replaces the seed.
- NO REGRESSION: ship the FusionSegmenter as a NEW registry segmenter, default-OFF (default\_segmitter unchanged); Gate A wiring goes behind indexing.min\_crossings defaulting to 0 so wasb\_hybrid/tracknet\_hybrid stay byte-identical today. Single-cue segmenters remain registered fallbacks.
- Court mask is an OPTIONAL config polygon (None => Gate B uses player-count only; supplied => masked). This subsumes both "manual polygon" and "count-only".
- Licensing MIT/Apache/BSD only; reuse PersonDetector (already extracted), rally\_gate (already exists), the trajectory runners, and the candidate stats persistence. Reuse, do not reinvent.
- Person features are few + scale/translation-invariant (small-N discipline).
- Maximize what is PURE-LOGIC unit-testable in a no-GPU dev container; clearly separate the GPU-only integration.

Output: (1) plan narrative; (2) files to add/edit with exact function signatures; (3)

config\_additions; (4) the pure-logic test plan; (5) gpu\_only items; (6) open\_questions. Prefer extending TrajectoryHybridSegmenter cleanly over copy-paste.

## 2. assistant (2026-06-13T18:49:19.679Z)

I'll synthesize a concrete plan from these subsystem maps. Let me first verify the key anchors against the actual code so the plan is buildable, not aspirational.

## 3. user (2026-06-13T18:49:23.098Z)

Already up to date.

## 4. assistant (2026-06-13T18:49:25.759Z)

Let me verify the key files and line anchors before building the plan.

## 5. user (2026-06-13T18:49:26.250Z)

```
1 """Shared engine for trajectory-substrate hybrid segmenters.
2
3 tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4 standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5 now the single home for the whole pipeline shape:
6
7 healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8 -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10 A concrete subclass supplies only its identity and runner wiring:
11 - ``family`` ? config section under ``indexing.*``, output subdir, and
12 the ``<family>-hybrid-<model>`` detector tag in metadata.
13 - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14 - ``name`` / ``description`` properties (the registry contract).
15 - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16 detection_params extras such as weights_path).
17
18 This base is deliberately NOT registered; only concrete subclasses register.
19 """
20 import os
21 import time
22 import logging
23 import concurrent.futures
24 from typing import Any, Dict, List, Optional, Tuple
25
26 import cv2
27
28 from backend.pipeline.segmenters.base import BasePlaySegmenter
29 from backend.utils.video import get_video_duration, extract_video_chunk
30 from backend.sports import sport_registry
31 from backend.providers import provider_registry
32 from backend.pipeline.detectors.base import DetectorRunner
33
34 logger = logging.getLogger(__name__)
35
36
37 def _fmt_ts(seconds: float) -> str:
38 seconds = max(0, int(seconds))
39 h, rem = divmod(seconds, 3600)
40 m, s = divmod(rem, 60)
41 return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44 class TrajectoryHybridSegmenter(BasePlaySegmenter):
45 """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46 boundaries."""
```

```

46
47 #: config section under `indexing` (velocity_threshold lives there), the
48 #: output subdir for trajectories, and the metadata detector-tag prefix.
49 family: str = ""
50 #: human-readable label used in log lines.
51 display_label: str = ""
52
53 def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54 """Return (runner, detector-specific detection_params extras)."""
55 raise NotImplementedError
56
57 @staticmethod
58 def _probe_fps_width(video_path: str) -> tuple:
59 """Return (fps, frame_width) for a video, with sensible fallbacks."""
60 cap = cv2.VideoCapture(video_path)
61 fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62 width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63 cap.release()
64 return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66 @staticmethod
67 def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68 """Provider-reported exchange count, else a duration-based estimate.
69
70 One canonical implementation ? the twins had drifted (wasb relied on
71 ``int(None)`` raising into the fallback; same result, by accident).
72 """
73 shot_count = segment.get("shuttle_exchange_count")
74 if shot_count is None:
75 return max(3, int(duration / 0.8))
76 try:
77 return int(shot_count)
78 except (ValueError, TypeError):
79 return max(3, int(duration / 0.8))
80
81 def process_video(self, video_id: str, video_path: str, # type: ignore[override]
82 player_pool: Optional[List[str]] = None,
83 max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
84 # override-ignore: the ABC declares the Result contract (Success|Failure)
85 # but every live segmenter still returns a bare list ? the documented
86 # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
87 label = self.display_label
88 # --- Sport profile + AI provider wiring (identical across segmenters) ---
89 sport_name = self.config.get("sport", "badminton")
90 try:
91 sport_profile = sport_registry.get(sport_name)
92 except KeyError as e:
93 logger.error(str(e))
94 return []
95
96 idx_cfg = self.config.get("indexing", {})
97 provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
98 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
99
100 api_key_env_var = f"{provider_name.upper()}_API_KEY"
101 api_key = os.environ.get(api_key_env_var)
102 if not api_key:
103 logger.error(
104 f"{api_key_env_var} environment variable is not set! "
105 f"To run the {provider_name} handoff, set your API key. "
106 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
107)

```



```

108 return []
109 try:
110 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
111 except KeyError as e:
112 logger.error(str(e))
113 return []
114
115 video_duration = get_video_duration(video_path)
116 if video_duration <= 0:
117 logger.error("Invalid video duration.")
118 return []
119 fps, frame_width = self._probe_fps_width(video_path)
120
121 # --- Runner config + windowing thresholds ---
122 runner, runner_params = self._build_runner(idx_cfg)
123
124 velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
125 merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
126 min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
127 handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
128 ai_max_workers = idx_cfg.get("ai_max_workers", 5)
129 log_candidates = idx_cfg.get("log_yolo_candidates", True)
130 store_candidates = idx_cfg.get("store_yolo_candidates", True)
131
132 detection_params = {
133 "detector": self.name,
134 "velocity_thresh": velocity_thresh,
135 "merge_gap": merge_gap,
136 "min_action_window_duration": min_window_duration,
137 **runner_params,
138 }
139
140 # --- Phase 1: env healthcheck (fail fast with a clear message) ---
141 ok, msg = runner.healthcheck()
142 if not ok:
143 logger.error("%s WSL environment not ready: %s", label, msg)
144 return []
145 logger.info("%s WSL env OK: %s", label, msg)
146
147 # --- Phase 2: trajectory -> candidate rally windows ---
148 # Trajectory detectors process the whole video in one pass (they carry
149 # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
150 t_start = time.time()
151 out_dir = os.path.join("output", self.family)
152 csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
153 if not csv_path:
154 logger.error("%s produced no trajectory; aborting.", label)
155 return []
156
157 points = runner.parse_trajectory_csv(csv_path)
158 logger.info("Parsed %d trajectory points (%d visible).",
159 len(points), sum(1 for p in points if p.visible))
160
161 all_candidate_windows = runner.trajectory_to_action_windows(
162 points, fps=fps, frame_width=frame_width, chunk_start=0.0,
163 velocity_thresh=velocity_thresh, merge_gap=merge_gap,
164 min_window_duration=min_window_duration, chunk_index=0,
165)
166 logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
167 label, time.time() - t_start, len(all_candidate_windows))
168
169 if log_candidates:
170 for cw in all_candidate_windows:
171 logger.info(f"[{label}] rally")

```

```

{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
172 f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
173 f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
174
175 # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
176 candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
177 if store_candidates:
178 for i, cw in enumerate(all_candidate_windows):
179 stats = {
180 "peak_velocity": cw["peak_velocity"],
181 "mean_velocity": cw["mean_velocity"],
182 "active_frame_count": cw["active_frame_count"],
183 "track_id_count": cw["track_id_count"],
184 }
185 candidate_ids[i] = self.db.add_candidate_segment(
186 video_id, detector=self.name,
187 start_time=cw["start"], end_time=cw["end"],
188 chunk_index=cw["chunk_index"], confidence=min(1.0,
189 cw["peak_velocity"]),
190 stats=stats, detection_params=detection_params,
191)
192 if not all_candidate_windows:
193 return []
194
195 # --- Phase 3: AI provider handoff over padded candidate clips ---
196 handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
197 os.makedirs(handoff_dir, exist_ok=True)
198 logger.info("Phase 3: %s validation on %d candidate windows...",
199 provider_name, len(all_candidate_windows))
200
201 def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
202 w_start = max(0, window["start"] - handoff_padding)
203 w_end = min(video_duration, window["end"] + handoff_padding)
204 w_dur = w_end - w_start
205 w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
206 if not extract_video_chunk(video_path, w_start, w_dur, w_path,
207 reencode=True):
208 return []
209 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
210 segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
211 label=f"Handoff {wi+1}")
212 valid = []
213 for seg in segments:
214 try:
215 seg["start_time"] = float(seg["start_time"]) + w_start
216 seg["end_time"] = float(seg["end_time"]) + w_start
217 seg["candidate_id"] = candidate_ids[wi]
218 valid.append(seg)
219 except (ValueError, TypeError, KeyError) as e:
220 logger.warning("Dropping segment with unparseable timestamps: %s ?
221 %s", seg, e)
222 try:
223 os.remove(w_path)
224 except OSError:
225 pass
226 return valid
227
228 ai_segments: List[dict] = []
229 with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
230 executor:
231 futures = [executor.submit(process_candidate_window, wi, cw)
232 for wi, cw in enumerate(all_candidate_windows)]

```

```

230 for future in futures:
231 ai_segments.extend(future.result())
232 try:
233 os.rmdir(handoff_dir)
234 except OSError:
235 pass
236 logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
237
238 # --- Phase 4: DB population + candidate linking ---
239 segments_data = []
240 for segment in ai_segments:
241 try:
242 start = float(segment["start_time"])
243 end = float(segment["end_time"])
244 dur = end - start
245 metadata = {
246 "shot_count": self._shot_count_from(segment, dur),
247 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
248 "detector": f"{self.family}-hybrid-{model_name}",
249 }
250 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
251 if segment_id:
252 candidate_id = segment.get("_candidate_id")
253 if candidate_id is not None:
254 self.db.link_candidate_to_segment(candidate_id, segment_id)
255 segments_data.append({
256 "id": segment_id, "start_time": start, "end_time": end,
257 "duration": dur, "metadata": metadata,
258 })
259 except Exception as e:
260 logger.warning("Skipping invalid segment struct: %s", e)
261
262 return segments_data
263

```

## 6. assistant (2026-06-13T18:49:29.378Z)

Now let me verify the rally\_gate, person\_detector, the registry, the config models, the Variant dataclass, and the existing tests in parallel.

## 7. user (2026-06-13T18:49:30.334Z)

```

1 """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3 WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4 the opponent for service after losing a point, that single trajectory looks like a
5 rally. The discriminator is structural, not duration-based: a real rally has the
6 shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7 single service feed crosses ~once.
8
9 This module is camera-agnostic and needs NO new model ? it works on the trajectory
10 we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11 axis with the larger shuttle spread) and the net position (median shuttle coord on
12 that axis ? the shuttle clusters at the two ends and crosses the net region often,
13 so the median sits near the net), then counts sign changes of (coord - net) across
14 the visible points inside a candidate window, with a deadband to ignore jitter.
15
16 Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17 Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18 windows from trajectory_to_action_windows, or to fold into it later.
19 """
20 from __future__ import annotations

```

```

21
22 from dataclasses import dataclass
23 from typing import List, Sequence, Tuple
24
25 Interval = Tuple[float, float]
26
27
28 @dataclass
29 class GatedWindow:
30 start: float
31 end: float
32 crossings: int
33 visible_points: int
34 kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38 """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39 if len(vals) < 2:
40 return 0.0
41 s = sorted(vals)
42 lo = s[max(0, int(0.05 * (len(s) - 1)))]
43 hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44 return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48 if not vals:
49 return 0.0
50 s = sorted(vals)
51 n = len(s)
52 return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56 """Return (axis 'x'/'y', net_value, span) from visible points.
57
58 Play axis = the image axis along which the shuttle travels most (larger spread).
59 For a baseline camera that's Y (players top/bottom); for a side camera, X.
60 """
61 xs = [p.x for p in points if p.visible]
62 ys = [p.y for p in points if p.visible]
63 sx, sy = _spread(xs), _spread(ys)
64 if sx >= sy:
65 return "x", _median(xs), sx
66 return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70 """Count sign changes of (coord - net) across visible points, ignoring the
71 deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72 crossings = 0
73 last_side = None
74 for p in window_points:
75 if not p.visible:
76 continue
77 v = (p.x if axis == "x" else p.y) - net
78 if abs(v) < deadband:
79 continue
80 side = v > 0
81 if last_side is not None and side != last_side:
82 crossings += 1
83 last_side = side
84 return crossings
85

```

```

86
87 def _points_in(points, fps: float, start: float, end: float):
88 return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92 min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93 """Annotate each (start,end) window with its shuttle net-crossing count and a
94 keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95 axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96 axis, net, span = estimate_play_axis(points)
97 deadband = deadband_frac * span
98 out: List[GatedWindow] = []
99 for (s, e) in windows:
100 wp = _points_in(points, fps, s, e)
101 c = count_net_crossings(wp, axis, net, deadband)
102 vis = sum(1 for p in wp if p.visible)
103 out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104 kept=c >= min_crossings))
105 return out
106
107
108 def apply_gate(points, windows: List[Interval], fps: float,
109 min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110 """Convenience: return only the windows that pass the crossing gate."""
111 return [(g.start, g.end) for g in
112 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113

```

## 8. user (2026-06-13T18:49:30.808Z)

```

1 """WASB-based hybrid play segmenter.
2
3 Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4 WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5 badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7 For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8 candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9 min_window_duration) are what get tuned against golden rally labels.
10
11 The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12 lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13 WASB runner and identity.
14 """
15 from typing import Any, Dict, Tuple
16
17 from backend.pipeline.segmenters.base import segmenter_registry
18 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19 from backend.pipeline.detectors.base import DetectorRunner
20 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23 class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24 family = "wasb"
25 display_label = "WASB"
26
27 @property
28 def name(self) -> str:
29 return "wasb_hybrid"
30
31 @property
32 def description(self) -> str:
33 return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained

```

```

badminton "
34 "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36 def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37 cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38 return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40
41 segmenter_registry.register(WasbHybridSegmenter)
42

```

## 9. user (2026-06-13T18:49:31.411Z)

```

1 """TrackNet-based hybrid play segmenter.
2
3 Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4 in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5 tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6 purpose-built trajectory model localises it far more precisely, giving cleaner
7 candidate rallies than person-velocity heuristics.
8
9 The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10 lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11 TrackNet runner and identity.
12 """
13 from typing import Any, Dict, Tuple
14
15 from backend.pipeline.segmenters.base import segmenter_registry
16 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
17 from backend.pipeline.detectors.base import DetectorRunner
18 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
19
20
21 class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
22 family = "tracknet"
23 display_label = "TrackNet"
24
25 @property
26 def name(self) -> str:
27 return "tracknet_hybrid"
28
29 @property
30 def description(self) -> str:
31 return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
32 "candidate rallies + AI provider for high-precision semantic
boundaries.")
33
34 def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
35 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
36 return TrackNetRunner(cfg), {
37 "weights_path": cfg.weights_path,
38 "queue_length": cfg.queue_length,
39 }
40
41
42 segmenter_registry.register(TrackNetHybridSegmenter)
43

```

## 10. assistant (2026-06-13T18:49:33.905Z)

Now the config models, the Variant/experiment harness, the person\_detector contract, and the

registry base.

## 11. user (2026-06-13T18:49:35.366Z)

```
1 """Pydantic models for the application configuration.
2
3 This module defines the canonical, validated, typed configuration for the
4 Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6 The models support graceful migration from old config.json files via extra="allow".
7 """
8
9 from __future__ import annotations
10
11 from typing import Any, List, Literal, Optional
12
13 from pydantic import BaseModel, Field, field_validator, model_validator
14
15 Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18 class ValidationRelevanceConfig(BaseModel):
19 court_green_lower: list[int] = Field(default=[35, 40, 40])
20 court_green_upper: list[int] = Field(default=[85, 255, 255])
21 court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24 class ValidationConfig(BaseModel):
25 min_resolution_width: int = 1280
26 min_resolution_height: int = 720
27 min_fps: float = 29.9
28 min_blur_threshold: float = 20.0
29 max_scene_cuts_per_minute: float = 0.5
30 relevance: ValidationRelevanceConfig =
31 Field(default_factory=ValidationRelevanceConfig)
32
33 class TrackNetConfig(BaseModel):
34 wsl_distro: str = "Ubuntu"
35 repo_dir: str = "~/models/TrackNetV4"
36 conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37 conda_env: str = "TrackNetV4"
38 weights_path: str = ""
39 queue_length: int = 5
40 stage_in_wsl: bool = True
41 wsl_stage_dir: str = "~/clips"
42 timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43 velocity_threshold: float = 0.02
44 # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45 # decoded frame cache) are deleted automatically ? they are pure intermediates,
46 # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47 # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48 keep_frames: bool = False
49 # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50 # 0 = disabled.
51 min_free_gb: float = 0.0
52
53
54 class WasbIndexConfig(BaseModel):
55 """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57 Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58 these knobs actually take effect ? previously this whole block was silently dropped
59 because nested config sections defaulted to extra='ignore'.
60 """
```

```

61 wsl_distro: str = "Ubuntu"
62 repo_dir: str = "~/models/WASB-SBDT"
63 conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64 conda_env: str = "wasb"
65 weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66 sport: str = "badminton"
67 wsl_stage_dir: str = "~/clips"
68 timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69 velocity_threshold: float = 0.02
70 # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71 # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72 # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73 keep_frames: bool = False
74 # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75 min_free_gb: float = 0.0
76
77
78 class IndexingConfig(BaseModel):
79 default_segmenter: str = "yolo_hybrid"
80 use_gpu: bool = True
81 motion_threshold: float = 0.08
82 min_segment_duration: float = 3.0
83 dead_time_merge_gap: float = 2.0
84 chunk_duration: float = 90.0
85 chunk_overlap: float = 10.0
86 detector_model: str = "fasterrcnn_resnet50_fpn"
87 detector_score_threshold: float = 0.5
88 yolo_velocity_threshold: float = 0.15
89 yolo_frame_skip: int = 3
90 yolo_tracker: str = "bytetrack.yaml"
91 yolo_prefetch_workers: int = 2
92 min_action_window_duration: float = 2.0
93 log_yolo_candidates: bool = True
94 store_yolo_candidates: bool = True
95 ai_handoff_padding: float = 2.0
96 ai_max_workers: int = 5
97 # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
98 # chunks of a high-bitrate (4K) source blow past the provider upload cap
99 # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
100 # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
101 # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
102 # Matches gemini_refine's --clip-scale-width default.
103 ai_chunk_scale_width: int = 1280
104 # Optional debug knob: trim every video to the first N seconds before processing.
105 debug_duration: Optional[float] = None
106
107 tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
108 wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
109
110 @field_validator("default_segmenter")
111 @classmethod
112 def segmenter_must_be_registered(cls, v: str) -> str:
113 # Registry check happens at runtime in main/segmenter selection.
114 # Here we just allow any string for forward compatibility.
115 return v
116
117 @model_validator(mode="after")
118 def _chunk_step_must_be_positive(self) -> "IndexingConfig":
119 # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
120 # step makes `while t < duration: t += step` loop forever, growing memory before

```



```

any
121 # GPU work. Reject the bad config at load time instead.
122 if self.chunk_overlap >= self.chunk_duration:
123 raise ValueError(
124 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
125)
126 f"({self.chunk_duration}); otherwise chunking never advances."
127)
128 return self
129
130 class OutputConfig(BaseModel):

```

## 12. user (2026-06-13T18:49:35.647Z)

```

1 """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3 This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4 identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5 no-regression gate, the pinnable classifier artifact, the persisted-candidate
6 substrate) already exists ? this module composes it. **No new scoring math lives
7 here**; everything defers to:
8
9 - `backend.eval.metrics` ? `score`, `match_intervals`, `interval_iou`
10 - `backend.eval.training` ? `label_candidates` (y by the same IoU matcher)
11 - `backend.eval.classifier` ? `LogisticRegression` (the pinnable JSON model)
12 - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13 - `backend.eval.regression` ? `gt_hash` (label-set fingerprint)
14 - `backend.eval.calibration` ? `pct_improvement`
15
16 The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17 (per-frame signals ? run once on the GPU/WSL box, persisted into
18 `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19 signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20 given persisted candidate features it produces `List[Interval]` deterministically,
21 with no GPU and zero production risk. So most experiments are pure offline
22 re-scoring of stored data and never touch the served pipeline.
23
24 Three modes (`EXPERIMENT_HARNESS.md` §5):
25 - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26 aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27 held-out loop, but variant-parameterised.
28 - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29 variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30 human spot-checks (and what two highlight reels would show side by side).
31 - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32 (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps

```

```

51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54
55 logger = logging.getLogger(__name__)
56
57 # Where a comparison run appends one reproducible record. Tracked location (NOT under
58 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
59 # regression.DEFAULT_BASELINE_PATH.
60 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
61 _METRICS = ("precision", "recall", "f1", "mean_iou")
62
63
64 class ExperimentError(ValueError):
65 """A variant cannot be evaluated as configured (e.g. a learned variant asked to
66 score an unlabeled video with no pinned model, or a gate referencing an absent
67 feature)."""
68
69
70 # ----- Variant
71
72
73 @dataclass
74 class Variant:
75 """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
76
77 A variant turns one video's persisted candidate windows + features into rally
78 intervals. Every knob defaults to the control behaviour (no gate, no learned
79 filter), so a variant that merely carries a new, disabled cue is byte-identical
80 to control ? the core no-regression guarantee.
81
82 Fields:
83 - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
84 for the leaderboard and for selecting the served path (the existing
85 ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
86 New cues are recorded here default-OFF.
87 - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
88 ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
89 ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
90 shuttle windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
91 - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
92 whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
93 cue persists that feature).
94 - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
95 ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
96 required for ``compare_unlabeled`` on a learned variant.
97 - ``feature_names`` ? the persisted features the learned filter consumes. When set
98 WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
99 (honest) ? the recipe, not a pinned artifact.
100 - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
101 overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
102 """
103
104 name: str
105 segmenter: str = "wasb_hybrid"
106 config_overrides: Dict[str, Any] = field(default_factory=dict)
107 cue_flags: Dict[str, bool] = field(default_factory=dict)
108 min_crossings: int = 0
109 min_players: int = 0
110 classifier_model: Optional[str] = None
111 feature_names: Optional[List[str]] = None
112 threshold: float = 0.5
113 merge_gap: float = 1.0
114

```

```

115 def is_learned(self) -> bool:
116 """True if this variant applies a learned classifier (pinned model or
recipe)."""
117 return self.classifier_model is not None or bool(self.feature_names)
118
119 def to_dict(self) -> dict:
120 return {
121 "name": self.name,
122 "segmenter": self.segmenter,
123 "config_overrides": self.config_overrides,
124 "cue_flags": self.cue_flags,
125 "min_crossings": self.min_crossings,
126 "min_players": self.min_players,
127 "classifier_model": self.classifier_model,
128 "feature_names": self.feature_names,
129 "threshold": self.threshold,
130 "merge_gap": self.merge_gap,
131 }
132
133 @classmethod
134 def from_dict(cls, d: dict) -> "Variant":
135 known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
136 return cls(**{k: v for k, v in d.items() if k in known})
137
138 def spec_hash(self) -> str:
139 """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
140 and makes a result reproducible. The pinned **control** variant always hashes
141 the
142 same, so a regression is always traceable to a recipe change, never to drift."""
143 blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
144 return sha1(blob.encode()).hexdigest()[:12]
145
146 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
147 #: filter. Always the comparison reference; "rollback" = make this the served variant.
148 CONTROL = Variant(name="control")
149
150
151 # ----- persisted candidates
152
153
154 @dataclass
155 class VideoCandidates:
156 """One video's persisted detection, ready for offline re-scoring.
157
158 ``intervals`` are the candidate windows; ``features`` is column-major
159 (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
160 ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
161 with ``load_video_candidates``, or directly in tests.
162 """
163
164 name: str
165 intervals: List[Interval]
166 features: Dict[str, List[float]] = field(default_factory=dict)
167 gts: Optional[List[Interval]] = None
168
169
170 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
171 """Persisted feature column, defaulting missing/short columns to 0.0 (matches
172 ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
173 col = vc.features.get(name)
174 n = len(vc.intervals)
175 if col is None:
176 logger.warning("variant references feature %r absent from %s ? treating as 0.0",
177 name, vc.name)

```

```

178 return [0.0] * n
179 if len(col) != n:
180 raise ExperimentError(
181 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
182 return [float(x) for x in col]
183
184
185 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
186 cols = [_feature_column(vc, name) for name in feature_names]
187 return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
188
189
190 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
191 """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
192 n = len(vc.intervals)
193 mask = [True] * n
194 if variant.min_crossings > 0:
195 col = _feature_column(vc, "net_crossings")
196 mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
197 if variant.min_players > 0:
198 col = _feature_column(vc, "players_in_court")
199 mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
200 return mask
201
202
203 def predict(variant: Variant, vc: VideoCandidates,
204 model: Optional[LogisticRegression] = None) -> List[Interval]:
205 """Variant prediction: gate by persisted features, optionally filter by a learned
206 model, then merge. Pure and deterministic.
207
208 ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
209 fold; if omitted and the variant pins ``classifier_model``, that frozen model is
210 loaded. A learned variant with neither a supplied nor a pinned model raises ? there
211 is nothing to score with.
212 """
213 mask = _gate_mask(variant, vc)
214 if variant.feature_names:
215 if model is None:
216 if variant.classifier_model is None:
217 raise ExperimentError(
218 f"variant {variant.name!r} is learned but has no model to predict "
219 f"with (pass a fitted model or set classifier_model)")
220 model = LogisticRegression.load(variant.classifier_model)
221 proba = model.predict_proba(_feature_matrix(vc, variant.feature_names))
222 mask = [m and (float(proba[i]) >= variant.threshold) for i, m in
enumerate(mask)]
223 kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
224 return merge_overlaps(kept, gap_tol=variant.merge_gap)
225
226
227 # ----- variant scoring
228
229
230 def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
231 """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
232 evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
233 X: List[List[float]] = []
234 y: List[int] = []
235 for vc in videos:
236 if vc.gts is None:
237 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")

```

```

238 X.extend(_feature_matrix(vc, variant.feature_names or []))
239 y.extend(label_candidates(vc.intervals, vc.gts, iou))
240 if not X:
241 raise ExperimentError("cannot train a learned variant on zero candidates")
242 return LogisticRegression().fit(X, y, variant.feature_names)
243
244
245 def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
246 iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
247 """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
248 mean aggregate.
249
250 Prediction policy:
251 - **static variant** (no learned filter): predict each video directly ? no
252 training, so no leakage; per-video scoring is already honest.
253 - **learned, pinned model** (`classifier_model` set): score every video with the
254 SHIPPED model. ? optimistic if those videos were in its training set ? use this
255 to report "how the shipped artifact does here", not for the promotion decision.
256 - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
257 ? train on the OTHER videos, predict the held-out one. The number to trust.
258 - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
259 only).
260
261 """
262 for vc in videos:
263 if vc.gts is None:
264 raise ExperimentError(f"{vc.name} has no golden labels; use
265 compare_unlabeled")
266
267 per_video: List[Dict[str, Any]] = []
268 predictions: Dict[str, List[Interval]] = {}
269
270 recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
271 pinned_model = (LogisticRegression.load(variant.classifier_model)
272 if variant.classifier_model is not None else None)
273 all_model = None
274 if recipe_lovo and not honest:
275 all_model = _train(variant, videos, iou)
276
277 for i, vc in enumerate(videos):
278 if recipe_lovo and honest:
279 others = [v for j, v in enumerate(videos) if j != i]
280 if not others:
281 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
282 pinned model")
283 model: Optional[LogisticRegression] = _train(variant, others, iou)
284 elif recipe_lovo:
285 # honest=False ? one model over all
286 videos
287 model = all_model
288 else:
289 # static variant or pinned model
290 model = pinned_model
291 preds = predict(variant, vc, model=model)
292 assert vc.gts is not None
293 # guarded at function top
294 sc = score(preds, vc.gts, iou)
295 per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
296 for m in _METRICS}})
297 predictions[vc.name] = preds
298
299 aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
300 0.0)
301 for m in _METRICS}
302 return {
303 "variant": variant.name,

```

```

294 "spec_hash": variant.spec_hash(),
295 "is_learned": variant.is_learned(),
296 "honest_lovo": recipe_lovo and honest,
297 "per_video": per_video,
298 "aggregate": aggregate,
299 "predictions": predictions,
300 }
301
302
303 def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
304 iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
305 """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
306 golden corpus and reports the **B ? A** deltas, per video and in aggregate.
307
308 The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
309 learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
310 `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
311 """
312 if len(videos) < 1:
313 raise ExperimentError("compare needs at least one labeled video")
314 eval_a = evaluate_variant(videos, variant_a, iou, honest)
315 eval_b = evaluate_variant(videos, variant_b, iou, honest)
316
317 delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
318 delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
319
320 by_video_a = {p["video"]: p for p in eval_a["per_video"]}
321 per_video_delta = [
322 {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
323 for p in eval_b["per_video"]
324]
325
326 # One fingerprint over the whole label set ? detects "the deltas were measured
against
327 # different labels" the same way regression.gt_hash guards the no-regression
baseline.
328 all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
329
330 return {
331 "iou": iou,
332 "label_set_hash": gt_hash(all_gts),
333 "num_videos": len(videos),
334 "variant_a": eval_a,
335 "variant_b": eval_b,
336 "delta": delta,
337 "delta_pct": delta_pct,
338 "per_video_delta": per_video_delta,
339 }
340
341
342 # ----- SxS on unlabeled video
343
344
345 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
346 agree_iou: float = 0.5) -> Dict[str, Any]:
347 """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
348
349 With no labels there is no lift to measure ? instead this surfaces where the two
350 variants **agree vs diverge**, which is exactly what a human reviews (and what two
351 highlight reels would show side by side):

```

```

352
353 - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
354 - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
355 - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
356 the human / a later golden label adjudicates).
357
358 Both variants must be predictable without labels: a learned variant therefore needs
a
359 pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
360 ``metrics.match_intervals`` ? no new matching logic.
361 """
362 preds_a = predict(variant_a, video)
363 preds_b = predict(variant_b, video)
364 mr = match_intervals(preds_a, preds_b, agree_iou)
365
366 agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
367 for m in mr.matches]
368 agree_ious = [m.iou for m in mr.matches]
369 a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
370 b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
371
372 def _dur(ivs: List[Interval]) -> float:
373 return sum(e - s for s, e in ivs)
374
375 return {
376 "video": video.name,
377 "agree_iou": agree_iou,
378 "variant_a": variant_a.name,
379 "variant_b": variant_b.name,
380 "num_a": len(preds_a),
381 "num_b": len(preds_b),
382 "num_agreed": len(agreed),
383 "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
384 "duration_a": _dur(preds_a),
385 "duration_b": _dur(preds_b),
386 "agreed": agreed,
387 "a_only": a_only,
388 "b_only": b_only,
389 }
390
391
392 # ----- leaderboard / DB
393
394
395 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
396 timestamp: Optional[str] = None) -> Dict[str, Any]:
397 """Append a compact, reproducible record of a ``compare()`` result to the JSON
398 leaderboard (``EXPERIMENT_HARNESS.md`` §6: experiments are records). Generalises
399 ``regression_baseline.json``; deliberately the size of a baseline file, not a service
400 (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
401 """
402 row: Dict[str, Any] = {
403 "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
404 "iou": result.get("iou"),
405 "label_set_hash": result.get("label_set_hash"),
406 "num_videos": result.get("num_videos"),
407 "variant_a": {"name": result["variant_a"]["variant"],
408 "spec_hash": result["variant_a"]["spec_hash"],
409 "aggregate": result["variant_a"]["aggregate"]},
410 "variant_b": {"name": result["variant_b"]["variant"],
411 "spec_hash": result["variant_b"]["spec_hash"],
412 "aggregate": result["variant_b"]["aggregate"]},
413 "delta": result.get("delta"),
414 }
415 entries: List[Dict[str, Any]] = []

```

```

416 if os.path.isfile(path):
417 with open(path, encoding="utf-8") as f:
418 entries = json.load(f)
419 entries.append(row)
420 os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
421 with open(path, "w", encoding="utf-8") as f:
422 json.dump(entries, f, indent=2)
423 logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
424 row["variant_a"]["name"], row["variant_b"]["name"],
425 (result.get("delta") or {}).get("f1", 0.0), path)
426 return row
427
428
429 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
430 gts: Optional[List[Interval]] = None,
431 feature_names: Sequence[str] = ()) -> VideoCandidates:
432 """Bridge persisted detection ? `VideoCandidates` via
433 `Database.query_candidate_segments`.
434 ``feature_names`` selects which persisted features to lift out of each row's
435 ``stats``
436 JSON (``duration`` is derived from the window bounds). This is the only DB-
437 touching
438 function; the comparison itself is pure and offline. Exercised owner-side on a real
439 DB,
440 but works against any `Database` (including an in-memory one in tests).
441 """
442 rows = db.query_candidate_segments(video_id, detector=detector)
443 intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
444 rows]
445 features: Dict[str, List[float]] = {}
446 for name in feature_names:
447 if name == "duration":
448 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
449 rows]
450 else:
451 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
452 rows]
453 return VideoCandidates(name=video_id, intervals=intervals, features=features,
454 gts=gts)
455
456

```

### 13. assistant (2026-06-13T18:49:40.308Z)

The experiment harness is the key insight: `Variant`, `\_gate\_mask`, and `predict` already implement Gate A (`min\_crossings` ? `net\_crossings`) and Gate B (`min\_players` ? `players\_in\_court`) as pure offline re-scoring. The DECISION layer already exists. What's missing is the DETECTION layer: persisting `net\_crossings` and the person features into `candidate\_segments.stats`, plus a `FusionSegmenter` that does the enrichment at index time. Let me verify the person\_detector contract, the segmenter registry/base, and existing tests.

### 14. user (2026-06-13T18:49:44.179Z)

```

1 """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3 Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4 the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5 `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
6 identically);
7 next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
8 later the learned classifier reads its persisted features. Keeping detection here (not
9 in
10 the segmenter) is what makes "add a cue = register a producer" true.
11
12 Behaviour is unchanged from the in-lined version ? only the home moved:

```



```

11 - `lazy_load_model` ? load a permissively-licensed torchvision detector
12 - `make_tracker` ? a fresh supervision ByteTrack per chunk
13 - `detect_and_track` ? one frame ? confident persons with persistent IDs
14 - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15 with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16 that feed the candidate `stats` JSON and the learned fusion features.
17
18 Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19 the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20 the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21 (ADR-005; ultralytics' AGPL YOLO was dropped).
22 """
23 import logging
24 from typing import Any, Dict, List, Optional, Tuple
25
26 import cv2
27 import torch
28
29 from backend.utils.geometry import get_velocity_normalised
30
31 logger = logging.getLogger(__name__)
32
33 # torchvision detection models are trained on COCO where the "person" category is
34 # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
35 # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
36 _PERSON_CLASS_ID = 1
37
38 # Permissively-licensed torchvision detectors selectable via config
39 # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
40 _DETECTOR_FACTORIES = {
41 "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
42 "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
43 "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
44 }
45 _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
46
47
48 class PersonDetector:
49 """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
50
51 Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
52 ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
53 min_action_window_duration}``); the model loads lazily on first use.
54 """
55
56 def __init__(self, config: Dict[str, Any]):
57 self.config = config
58 # Dynamic torchvision module (None until lazy_load_model); typed Any so the
59 # `self.model([tensor])` inference call type-checks without a quarantine.
60 self.model: Any = None
61 self.device: Optional[str] = None
62
63 def lazy_load_model(self) -> None:
64 if self.model is not None:
65 return
66
67 # Imported lazily so the module imports without torchvision present and so
68 # the test suite (which pre-sets self.model) never pulls in model weights.
69 from torchvision.models import detection as tv_detection
70
71 idx_cfg = self.config.get("indexing", {})
72 use_gpu = idx_cfg.get("use_gpu", True)
73 self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74 if use_gpu and not torch.cuda.is_available():

```

```

75 logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77 model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78 if model_name not in _DETECTOR_FACTORIES:
79 logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80 model_name = _DEFAULT_DETECTOR
81
82 builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83 logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84 # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85 self.model = builder(weights="DEFAULT")
86 self.model.eval()
87 self.model.to(self.device)
88
89 def make_tracker(self, fps: float) -> Any:
90 """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92 Imported lazily so tests can mock this method without supervision installed.
93 """
94 import supervision as sv
95 frame_rate = max(1, int(round(fps)))
96 return sv.ByteTrack(frame_rate=frame_rate)
97
98 def detect_and_track(self, frame: Any, tracker: Any,
99 score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100 """Detect persons in a frame and assign persistent track IDs.
101
102 Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103 This replaces the old ultralytics ``model.track(...)`` call, which bundled
104 detection AND association: torchvision does the detection, supervision's
105 ByteTrack does the association.
106 """
107 import supervision as sv
108
109 # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110 rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111 tensor = torch.from_numpy(rgb).permute(2, 0,
112 1).float().div(255.0).to(self.device)
113 with torch.no_grad():
114 outputs = self.model([tensor])
115
116 out = outputs[0]
117 boxes = out["boxes"].detach().cpu().numpy()
118 labels = out["labels"].detach().cpu().numpy()
119 scores = out["scores"].detach().cpu().numpy()
120
121 # Keep only confident person detections.
122 mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
123 if not mask.any():
124 return []
125
126 # ByteTrack requires confidence to be populated (it filters on it internally).
127 dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
128 tracked = tracker.update_with_detections(dets)
129
130 result: List[Tuple[int, Tuple[float, float, float, float]]] = []
131 for i in range(len(tracked)):
132 tid = tracked.tracker_id[i]
133 if tid is None:
134 continue
135 x1, y1, x2, y2 = tracked.xyxy[i]

```

```

135 result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136 return result
137
138 def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139 velocity_thresh: float, merge_gap: float,
140 frame_skip: int = 3,
141 chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142 """
143 Runs person detection + tracking on a video chunk and extracts high-motion
144 action windows. Returns a list of dicts in the full video's timeframe, each
with:
145 start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146 track_id_count, chunk_index.
147
148 Args:
149 frame_skip: Process every Nth frame (1 = every frame). Higher values
150 dramatically reduce inference cost with minimal quality loss.
151 chunk_index: Index of the source chunk (recorded on each window for
traceability).
152 """
153 cap = cv2.VideoCapture(chunk_path)
154 if not cap.isOpened():
155 logger.error(f"Could not open {chunk_path}")
156 return []
157
158 fps = cap.get(cv2.CAP_PROP_FPS)
159 if fps <= 0:
160 fps = 30.0
161
162 frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163 if frame_width <= 0:
164 frame_width = 1280.0 # Sensible fallback
165
166 # Effective sampling rate after frame-skipping
167 effective_fps = fps / max(frame_skip, 1)
168
169 # A fresh tracker per chunk ? track IDs only need to be consistent within a
170 # chunk (windows are computed per chunk, then merged across chunks by time).
171 tracker = self.make_tracker(effective_fps)
172 score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174 # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175 # so windows can be enriched with motion stats for logging + fine-tuning.
176 active_frames = []
177 frame_idx = 0
178 track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180 while True:
181 ret, frame = cap.read()
182 if not ret:
183 break
184
185 # Skip frames for performance ? at 30fps, every 3rd frame still
186 # gives ~10 samples/second, plenty for human-scale velocity.
187 if frame_idx % max(frame_skip, 1) != 0:
188 frame_idx += 1
189 continue
190
191 # Detect persons + assign persistent track IDs.
192 detections = self.detect_and_track(frame, tracker, score_thresh)
193
194 frame_peak_velocity = 0.0
195 frame_active_ids = set()

```

```

196 current_ids = set()
197
198 for track_id, bbox in detections:
199 current_ids.add(track_id)
200
201 if track_id in track_history:
202 prev_bbox = track_history[track_id]
203 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204 if velocity > velocity_thresh:
205 frame_peak_velocity = max(frame_peak_velocity, velocity)
206 frame_active_ids.add(track_id)
207
208 track_history[track_id] = bbox
209
210 # Prune stale IDs that are no longer visible ? prevents false
211 # velocity spikes when the tracker reassigns an old ID.
212 track_history = {k: v for k, v in track_history.items() if k in current_ids}
213
214 if frame_active_ids:
215 active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
216
217 frame_idx += 1
218
219 cap.release()
220
221 # Group active frames into continuous windows
222 if not active_frames:
223 return []
224
225 max_gap_frames = int(merge_gap * fps)
226
227 def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
228 velocities = [v for _, v, _ in group]
229 track_ids = set().union(*[ids for _, _, ids in group])
230 return {
231 "start": group[0][0] / fps + chunk_start,
232 "end": group[-1][0] / fps + chunk_start,
233 "active_frame_count": len(group),
234 "peak_velocity": max(velocities),
235 "mean_velocity": sum(velocities) / len(velocities),
236 "track_id_count": len(track_ids),
237 "chunk_index": chunk_index,
238 }
239
240 windows = []
241 group = [active_frames[0]]
242 for af in active_frames[1:]:
243 if af[0] - group[-1][0] <= max_gap_frames:
244 group.append(af)
245 else:
246 windows.append(_finalize(group))
247 group = [af]
248 windows.append(_finalize(group))
249
250 # Filter out extremely short action windows
251 min_window_duration = self.config.get("indexing",
{}).get("min_action_window_duration", 2.0)
252 windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
253
254 return windows
255

```

```

1 from abc import ABC, abstractmethod
2 from typing import List, Dict, Any, Type, Optional
3 from backend.infrastructure.database import Database
4 from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6 class BasePlaySegmenter(ABC):
7 def __init__(self, db: Database, config: Dict[str, Any]):
8 self.db = db
9 self.config = config
10
11 @property
12 @abstractmethod
13 def name(self) -> str:
14 """Unique identifier for this segmenter."""
15 pass
16
17 @property
18 @abstractmethod
19 def description(self) -> str:
20 """Human-readable explanation of this segmenter."""
21 pass
22
23 @abstractmethod
24 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25 """
26 Processes a video to detect play segments, populates SQLite database,
27 and returns a Result.
28
29 Use isinstance(result, Failure) (or result.is_success()) to distinguish
30 "no segments found" from "something went wrong" (missing key, healthcheck
31 failure, etc.). This is the start of the standardized error/result contract.
32 """
33 pass
34
35 class SegmenterRegistry:
36 def __init__(self):
37 self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39 def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40 """Registers a new segmenter class under its name property."""
41 try:
42 temp_inst = segmenter_cls(None, {})
43 name = temp_inst.name
44 except Exception as e:
45 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46 self._segmenters[name] = segmenter_cls
47
48 def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49 return self._segmenters.get(name)
50
51 def list_available(self) -> Dict[str, str]:
52 """Returns a dict of registered segmenter names and their descriptions."""
53 res = {}
54 for name, cls in self._segmenters.items():
55 try:
56 temp_inst = cls(None, {})
57 desc = temp_inst.description
58 except Exception:
59 desc = "No description available."
60 res[name] = desc
61 return res
62

```

```
63 segmenter_registry = SegmenterRegistry()
64
```

## 16. assistant (2026-06-13T18:49:47.329Z)

Now let me check the database `add\_candidate\_segment` signature, the existing tests, the distill\_local FEATURE\_NAMES, and the tracknet\_runner trajectory\_to\_action\_windows + TrajectoryPoint, plus geometry helpers.

## 17. user (2026-06-13T18:49:48.714Z)

```
230 conn.cursor().execute(
231 "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
232 (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
233)
234 conn.commit()
235 return True
236 except Exception as e:
237 logger.error(f"Failed to add event: {e}")
238 return False
239
240 def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
241 chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
242 stats: Optional[Dict[str, Any]] = None,
243 detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
244 """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
245 detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
246 try:
247 with self._connect() as conn:
248 cursor = conn.cursor()
249 cursor.execute(
250 """INSERT INTO candidate_segments
251 (video_id, detector, chunk_index, start_time, end_time, duration,
252 confidence, stats, detection_params)
253 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
254 (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
255 confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
256)
257 conn.commit()
258 return cursor.lastrowid
259 except Exception as e:
260 logger.error(f"Failed to add candidate segment: {e}")
261 return None
262
263 def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
264 """Record that a candidate detection was confirmed as a given play_segment."""
265 try:
266 with self._connect() as conn:
267 conn.cursor().execute(
268 "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
269 (segment_id, candidate_id)
270)
271 conn.commit()
272 return True
273 except Exception as e:
274 logger.error(f"Failed to link candidate {candidate_id} to segment
```

```
{segment_id}: {e}")
```

#### 18. user (2026-06-13T18:49:49.661Z)

```
30 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31 from backend.pipeline.detectors import rally_gate
32 from backend.eval.calibrate_wasb import load_golden_gts
33 from backend.eval.training import label_candidates
34 from backend.eval.classifier import LogisticRegression
35 from backend.eval.metrics import score
36 from backend.eval.gemini_refine import merge_overlaps
37
38 Interval = Tuple[float, float]
39
40 FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
41 "net_crossings"]
42 # Recall-oriented proposal: deliberately permissive so real rallies are among the
43 # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
44 PROPOSAL = (0.005, 1.0, 0.5)
45 BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
46
47 def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
48 proposal: Tuple[float, float, float] = PROPOSAL) ->
49 Tuple[List[Interval], List[List[float]]]:
50 """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
51 rows)."""
52 points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
53 vt, mg, mwd = proposal
54 wins = TrackNetRunner.trajectory_to_action_windows(
55 points, fps=fps, frame_width=frame_width,
56 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
57 intervals = [(w["start"], w["end"]) for w in wins]
58 gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
59 crossings
60 X: List[List[float]] = []
61 for w, g in zip(wins, gated):
62 dur = max(1e-6, w["end"] - w["start"])
63 win_frames = max(1.0, dur * fps)
64 X.append([
65 dur, # rally length (sec) ?
66 float(w["peak_velocity"]), # already normalized by
67 float(w["mean_velocity"]),
68 float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
69 float(g.crossings), # net crossings (count) ?
70 the rally signal
71])
72 return intervals, X
73
74
75
76
77
78
79 @dataclass
80 class TrajectoryPoint:
81 frame: int
82 visible: bool
83 x: float
84 y: float
```

#### 19. user (2026-06-13T18:49:50.606Z)

```
75 return f"/mnt/{drive}{p[2:]}"
76 return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81 frame: int
82 visible: bool
83 x: float
84 y: float
```

```

85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88 """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89

```

## 20. user (2026-06-13T18:49:51.203Z)

```

1 import math
2 from typing import Tuple
3
4 def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
5 """Calculate the center (x, y) of a bounding box.
6 bbox format: (x1, y1, x2, y2)
7 """
8 x1, y1, x2, y2 = bbox
9 return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)
10
11 def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
12 """Calculate the Euclidean distance between two points (x, y)."""
13 return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)
14
15 def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
float, float], fps: float) -> float:
16 """Calculate velocity (pixels per second) of a bounding box moving between two
consecutive frames.
17 """
18 c1 = get_center(bbox1)
19 c2 = get_center(bbox2)
20 dist = get_distance(c1, c2)
21 return dist * fps
22
23 def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
Tuple[float, float, float, float],
24 fps: float, frame_width: float) -> float:
25 """Calculate velocity as a fraction of frame width per second.
26
27 This makes the threshold resolution-independent:
28 - 0.15 means the center moved 15% of the frame width in one second.
29 - Works identically on 720p, 1080p, and 4K footage.
30
31 Args:
32 bbox1: Previous bounding box (x1, y1, x2, y2).
33 bbox2: Current bounding box (x1, y1, x2, y2).
34 fps: Effective frames per second (after frame-skipping).
35 frame_width: Width of the video frame in pixels.
36 """
37 raw_velocity = get_velocity(bbox1, bbox2, fps)
38 if frame_width <= 0:
39 return raw_velocity # Fallback to raw pixels if width unknown
40 return raw_velocity / frame_width
41
42 def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
float, float]) -> float:
43 """Calculate the Intersection over Union (IoU) of two bounding boxes.
44 bbox format: (x1, y1, x2, y2)
45 """
46 xA = max(boxA[0], boxB[0])
47 yA = max(boxA[1], boxB[1])
48 xB = min(boxA[2], boxB[2])
49 yB = min(boxA[3], boxB[3])
50
51 interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
52 if interArea == 0:
53 return 0.0

```



```

54
55 boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
56 boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)
57
58 iou = interArea / float(boxAArea + boxBArea - interArea)
59 return iou
60

```

## 21. assistant (2026-06-13T18:49:54.193Z)

Now let me look at the existing tests for the equivalence/regression contracts and how the trajectory runner exposes `parse\_trajectory\_csv` (to reuse shuttle points for colocation), plus check the test directory layout.

## 22. user (2026-06-13T18:49:55.633Z)

```

232 @staticmethod
233 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
234 """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
235 points: List[TrajectoryPoint] = []
236 with open(csv_win_path, newline="") as f:
237 reader = csv.DictReader(f)
238 for row in reader:
239 try:
240 points.append(TrajectoryPoint(
241 frame=int(row["Frame"]),
242 visible=int(row["Visibility"]) == 1,
243 x=float(row["X"]),
244 y=float(row["Y"]),
245))
246 except (KeyError, ValueError):
247 continue
248 points.sort(key=lambda p: p.frame)
249 return points
250
251 @staticmethod
252 def trajectory_to_action_windows(
253 points: List[TrajectoryPoint],
254 fps: float,
255 frame_width: float,
256 chunk_start: float = 0.0,
257 velocity_thresh: float = 0.02,
258 merge_gap: float = 2.0,
259 min_window_duration: float = 2.0,
260 chunk_index: Optional[int] = None,
261) -> List[Dict[str, Any]]:
262 """Convert a shuttle trajectory into candidate rally windows.
263
264 A frame is "active" when the shuttle is visible AND its inter-frame
265 displacement (normalised by frame width, per second) exceeds
266 ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
267 ``merge_gap`` seconds of each other are grouped into one window; short
268 windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
269 shape feeding the AI-handoff phase is identical.
270 """
271 if fps <= 0:
272 fps = 30.0
273 if frame_width <= 0:
274 frame_width = 1280.0
275
276 # Compute per-frame normalised velocity from the last *visible* point.
277 active = [] # (frame_idx, velocity)
278 prev: Optional[TrajectoryPoint] = None
279 for p in points:
280 if not p.visible:

```

```

281 prev = None # break the trajectory; absent shuttle = not in flight
282 continue
283 if prev is not None:
284 dframes = p.frame - prev.frame
285 if dframes > 0:
286 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287 velocity = (dist / frame_width) * (fps / dframes)
288 if velocity > velocity_thresh:
289 active.append((p.frame, velocity))
290 prev = p
291
292 if not active:
293 return []
294
295 max_gap_frames = int(merge_gap * fps)
296
297 def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
298 vels = [v for _, v in group]
299 return {
300 "start": group[0][0] / fps + chunk_start,
301 "end": group[-1][0] / fps + chunk_start,
302 "active_frame_count": len(group),
303 "peak_velocity": max(vels),
304 "mean_velocity": sum(vels) / len(vels),
305 "track_id_count": 1, # single shuttle; kept for window-shape parity
306 "chunk_index": chunk_index,
307 }
308
309 windows = []
310 group = [active[0]]
311 for af in active[1:]:
312 if af[0] - group[-1][0] <= max_gap_frames:
313 group.append(af)
314 else:
315 windows.append(_finalize(group))
316 group = [af]
317 windows.append(_finalize(group))
318
319 return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320

```

### 23. user (2026-06-13T18:49:55.861Z)

```

tests__init__.py
tests\test_eval_batch.py
tests\test_eval_training.py
tests\test_annotation_providers.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\test_export_wasb_segments.py
tests\test_gemini_refine.py
tests\test_calibrate_local.py
tests\test_distill_local.py
tests\test_automated_provider.py
tests\test_eval_harness.py
tests\test_hit_detector.py
tests\test_rally_gate.py
tests\test_validators_base.py
tests\test_hashing.py
tests\test_rally_slicer.py
tests\test_reporting_spec_faq.py
tests\test_tracknet_runner.py
tests\test_eval_partial_labels.py
tests\test_detector_keying.py
tests\test_reingest_safety.py

```

```
tests\test_eval_metrics.py
tests\test_compiler.py
tests\test_eval_annotations.py
tests\test_gt_loader.py
tests\test_import_smoke.py
tests\test_config_wasb.py
tests\test_reporting_harvest.py
tests\test_run_signals.py
tests\test_async_jobs.py
tests\test_base.py
tests\test_database.py
tests\test_eval_classifier.py
tests\test_eval_regression.py
tests\test_gemini.py
tests\test_geometry.py
tests\test_path_safety.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_rally_seq_proto.py
tests\test_reliability_hardening.py
tests\test_reporting.py
tests\test_sport_profiles.py
tests\test_training_gen0.py
tests\test_video.py
tests\test_wasb_cache.py
tests\test_config.py
tests\test_tracknet_hybrid.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_wsl_tilde_quoting.py
tests\test_upload_download.py
tests\test_api_endpoints.py
tests\test_watermark.py
tests\test_cleanup_caches.py
tests\test_wasb_runner.py
tests\test_experiment.py
tests\test_person_detector.py
tests\test_yolo_hybrid.py
```

#### 24. assistant (2026-06-13T18:49:59.276Z)

Let me look at the existing equivalence test and the experiment test to anchor the regression test plan precisely, and confirm how `run\_predict` returns the CSV (so the FusionRunner can reuse the trajectory CSV for both shuttle points and colocation).

#### 25. user (2026-06-13T18:50:00.291Z)

```
1 """Equivalence contract for the trajectory-hybrid twins.
2
3 tracknet_hybrid and wasb_hybrid were copy-paste twins that drifted (the
4 shot_count parse); both are now thin subclasses of TrajectoryHybridSegmenter.
5 These tests run BOTH through the same scenarios and assert identical P3/P4
6 behavior, so any future divergence has to be introduced deliberately, in the
7 base, for both at once.
8 """
9 from unittest.mock import MagicMock, patch
10
11 import pytest
12
13 from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
14 from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
15 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
16
17 TWINS = [
18 (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
19 (WasbHybridSegmenter, "wasb", "wasb_hybrid"),
```

```

20]
21
22
23 def _window(start=1.0, end=4.0):
24 return {
25 "start": start, "end": end, "active_frame_count": 30,
26 "peak_velocity": 0.4, "mean_velocity": 0.2, "track_id_count": 1,
27 "chunk_index": 0,
28 }
29
30
31 def _runner(windows=None):
32 runner = MagicMock()
33 runner.healthcheck.return_value = (True, "env ok")
34 runner.run_predict.return_value = "out.csv"
35 runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
36 runner.trajectory_to_action_windows.return_value = windows if windows is not None
37 else [_window()]
38 return runner
39
40 def _run(cls, provider_segments, db=None):
41 """Run one segmenter class through the standard mocked pipeline."""
42 provider = MagicMock()
43 provider.analyze_video.return_value = (provider_segments, {})
44 db = db or MagicMock()
45 db.add_candidate_segment.return_value = 55
46 db.add_play_segment.return_value = 200
47
48 with patch.object(cls, "_build_runner", return_value=(_runner(), {"weights_path":
"w"})), \
49 patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), \
50 patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
51 patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
52 patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
53 patch("os.environ.get", return_value="dummy_key"):
54 preg.get.return_value = provider
55 seg = cls(db, {"indexing": {}})
56 return db, seg.process_video("v1", "clip.mp4")
57
58
59 @pytest.mark.parametrize("cls,family,detector", TWINS)
60 def test_detector_identity_flows_into_db_rows(cls, family, detector):
61 db, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count":
4}])
62 assert len(res) == 1
63 _, kwargs = db.add_candidate_segment.call_args
64 assert kwargs["detector"] == detector
65 assert kwargs["detection_params"]["detector"] == detector
66 # P4 metadata tags the family + model.
67 assert res[0]["metadata"]["detector"].startswith(f"{family}-hybrid-")
68 db.link_candidate_to_segment.assert_called_once_with(55, 200)
69
70
71 @pytest.mark.parametrize("cls,family,detector", TWINS)
72 @pytest.mark.parametrize("exchange,expected", [
73 (4, 4), # provider-reported count wins
74 ("7", 7), # numeric strings coerce
75 (None, 3), # explicit null -> duration estimate (2.5s/0.8 -> 3 floor)
76 ("garbage", 3), # unparseable -> same estimate
77])
78 def test_shot_count_behavior_is_identical(cls, family, detector, exchange, expected):

```

```

79 """The historical drift point: wasb_hybrid relied on int(None) raising into
80 the fallback while tracknet_hybrid short-circuited. One implementation now."""
81 seg_dict = {"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": exchange}
82 _, res = _run(cls, [seg_dict])
83 assert res[0]["metadata"]["shot_count"] == expected
84
85
86 @pytest.mark.parametrize("cls, family, detector", TWINS)
87 def test_provider_offsets_applied_identically(cls, family, detector):
88 # Window [1,4] + 2.0s default padding -> clip starts at max(0, -1) = 0.0;
89 # provider-relative 0.5 stays 0.5 absolute here.
90 _, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
91 assert res[0]["start_time"] == 0.5
92 assert res[0]["end_time"] == 3.0
93
94
95 def test_base_is_not_registered():
96 from backend.pipeline.segmenters.base import segmenter_registry
97 available = segmenter_registry.list_available()
98 assert "tracknet_hybrid" in available
99 assert "wasb_hybrid" in available
100 # The base engine itself must not appear as a selectable segmenter.
101 assert not any(isinstance(cls, type) and cls is TrajectoryHybridSegmenter
102 for cls in available.values())
103

```

## 26. assistant (2026-06-13T18:50:03.770Z)

I now have everything I need. Let me check the `test\_experiment.py` for the exact byte-identity test, and confirm there is a hook in `process\_video` where I can insert enrichment without touching the immutable flow. The cleanest seam is an overridable hook method.

## 27. user (2026-06-13T18:50:04.732Z)

```

1 """Tests for the experiment harness (backend.eval.experiment).
2
3 Pure/offline ? synthetic candidate rows, no GPU/WSL. The load-bearing test is the
4 **no-regression invariant** (`test_disabled_cues_byte_identical_to_control`): a variant
5 that merely carries a disabled cue must predict exactly what control predicts. That is
the
6 guarantee that lets new cues be merged freely (`EXPERIMENT_HARNESS.md` §6).
7 """
8 import json
9
10 import pytest
11
12 from backend.eval.classifier import LogisticRegression
13 from backend.eval.experiment import (
14 CONTROL,
15 ExperimentError,
16 Variant,
17 VideoCandidates,
18 compare,
19 compare_unlabeled,
20 evaluate_variant,
21 load_video_candidates,
22 predict,
23 record_experiment,
24)
25 from backend.infrastructure.database import Database
26
27
28 # A video whose third candidate is a held-shuttle false fire (net_crossings=0): it does
29 # not overlap any golden rally, so it is a negative by construction.
30 def _held_shuttle_video(name="v", gts=None):

```

```

31 return VideoCandidates(
32 name=name,
33 intervals=[(0.0, 10.0), (20.0, 30.0), (12.0, 13.0)],
34 features={"net_crossings": [4.0, 3.0, 0.0]},
35 gts=gts if gts is not None else [(0.0, 10.0), (20.0, 30.0)],
36)
37
38
39 # Three videos where feature "x" perfectly separates rallies (x=5) from junk (x=0),
40 # so an honest LOVO learned variant should recover F1?1.0 on each held-out video.
41 def _separable_videos():
42 vids = []
43 for k in range(3):
44 base = 100.0 * k
45 vids.append(VideoCandidates(
46 name=f"sep{k}",
47 intervals=[(base + 0, base + 10), (base + 20, base + 30),
48 (base + 40, base + 41), (base + 50, base + 51)],
49 features={"x": [5.0, 5.0, 0.0, 0.0]},
50 gts=[(base + 0, base + 10), (base + 20, base + 30)],
51))
52 return vids
53
54
55 # ----- Variant
56
57 def test_variant_roundtrip_and_hash_stability():
58 v = Variant(name="gateA", min_crossings=2, cue_flags={"person": False})
59 assert Variant.from_dict(v.to_dict()) == v
60 assert v.spec_hash() == v.spec_hash() # stable
61 assert CONTROL.spec_hash() == Variant(name="control").spec_hash()
62 assert v.spec_hash() != CONTROL.spec_hash() # different recipe ?
different hash
63
64
65 def test_from_dict_ignores_unknown_keys():
66 v = Variant.from_dict({"name": "x", "min_crossings": 1, "bogus": 99})
67 assert v.name == "x" and v.min_crossings == 1
68
69
70 def test_is_learned():
71 assert not CONTROL.is_learned()
72 assert Variant(name="r", feature_names=["x"]).is_learned()
73 assert Variant(name="p", classifier_model="model.json").is_learned()
74
75
76 # ----- the no-regression invariant
77
78 def test_disabled_cues_byte_identical_to_control():
79 """A variant carrying only DISABLED cues must equal control, window-for-window."""
80 vc = _held_shuttle_video()
81 disabled = Variant(name="fusion_off", cue_flags={"person": False, "audio": False},
82 min_crossings=0, min_players=0)
83 assert predict(disabled, vc) == predict(CONTROL, vc)
84
85
86 def test_control_is_merged_candidates():
87 vc = VideoCandidates(name="v", intervals=[(0.0, 10.0), (10.5, 12.0), (40.0, 50.0)])
88 # merge_gap=1.0 stitches the 0.5s gap between the first two; the 28s gap stays
split.
89 assert predict(CONTROL, vc) == [(0.0, 12.0), (40.0, 50.0)]
90
91
92 # ----- decision gates
93

```

```

94 def test_gate_a_drops_low_crossing_window():
95 vc = _held_shuttle_video()
96 gate_a = Variant(name="gateA", min_crossings=2)
97 preds = predict(gate_a, vc)
98 assert (12.0, 13.0) not in preds # held-shuttle fire
removed
99 assert preds == [(0.0, 10.0), (20.0, 30.0)]
100 assert preds != predict(CONTROL, vc) # the gate actually
changed output

```

## 28. assistant (2026-06-13T18:50:07.689Z)

I have complete verification of all anchors. The architecture is clear. Let me run the existing test suite quickly to confirm the baseline is green before I propose changes (so the plan's regression guard is grounded).

## 29. user (2026-06-13T18:50:17.331Z)

```

..... [100%]
48 passed in 6.13s

```

## 30. assistant (2026-06-13T18:50:24.759Z)

Baseline is green. I now have full verification of every seam. Let me synthesize the plan.

Key findings that shape the design:

1. **The DECISION layer already exists** ? ``experiment.Variant` + `_gate_mask` + `predict`` already implement Gate A (``min_crossings`?`net_crossings``) and Gate B (``min_players`?`players_in_court``) as pure offline re-scoring, default-OFF. What's missing is the **DETECTION layer**: persisting those features into ``candidate_segments.stats`` at index time.
2. **The cleanest no-regression seam** is a single overridable hook in the immutable ``process_video`` flow at the stats-building step (lines 179-184). The base hook is identity; ``FusionSegmenter`` overrides it to enrich. This avoids touching the Phase 2/3/4 control flow entirely.
3. **`net\_crossings` is pure-logic and GPU-free** ? ``rally_gate.gate_windows`` already runs on trajectory points we always have. So Gate-A feature persistence can ship for ALL trajectory segmenters with zero GPU, behind ``min_crossings`` default 0.
4. Person features need GPU (torchvision) ? gated behind ``cue_flags["person"]``, default OFF, only instantiated when enabled.

## 31. user (2026-06-13T18:51:59.910Z)

Structured output provided successfully

## 32. assistant (2026-06-13T18:52:14.149Z)

I have synthesized and verified the buildable Fusion P2 plan against the actual codebase. Summary of what I confirmed and delivered:

**Key load-bearing discovery:** The DECISION layer is already shipped in ``backend/eval/experiment.py`` ? ``Variant.min_crossings`?`net_crossings`` (Gate A) and ``Variant.min_players`?`players_in_court`` (Gate B) are already wired through ``_gate_mask`/`predict``, default-OFF, with the byte-identity regression test (``test_disabled_cues_byte_identical_to_control``) already passing. P2 is therefore almost entirely the missing DETECTION layer: persisting those features at index time. This collapses scope and de-risks the whole effort.

**The no-regression seam:** One overridable hook ``_enrich_candidate_stats(...)`` inserted at ``trajectory_hybrid.py:179-184``, whose base body returns the exact current 4-key stats dict ? so `wasb_hybrid/tracknet_hybrid` stay byte-identical, the immutable Phase 1-4 flow is untouched, and ``detection_params`` is never modified (LOVO reproducibility preserved). `FusionSegmenter` is the only override.

**\*\*Verified all anchors\*\*** against real code: the engine flow (81?262), ``add_candidate_segment`` signature, ``rally_gate.gate_windows`` (GPU-free ``net_crossings``), ``PersonDetector.extract_action_windows_from_chunk``, the segmenter registry, config models, ``Variant`/`predict``, ``distill_local.FEATURE_NAMES``, and ``TrajectoryPoint``. Ran the 5 relevant test suites green (48 passed) to ground the regression baseline.

The structured output contains the full plan narrative, 7 files (2 edits to engine/config, 1 edit to feature-names, 3 new modules/tests, plus the equivalence-test extension) with exact function signatures, 8 config additions, a 14-item pure-logic test plan, 6 GPU-only items cleanly separated, and 5 open questions (the most material being ``frame_height`` probing for court-polygon y-normalization, and whether P2b computes true per-frame person features or window-level approximations).

Relevant files (absolute):

- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\rally_gate.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\person_detector.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\experiment.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py``